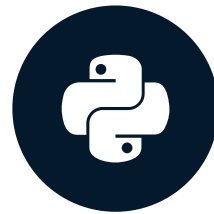


What is financial trading

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

The concept of financial trading

Financial trading is the buying and selling of financial assets

Various financial instruments to trade:

- Equities
- Bonds
- Forex
- Commodities
- Cryptocurrencies

Why people trade

To make a profit by taking calculated risks

- Long positions: profit from upward price movement
- Short positions: profit from downward price movement

Market participants:

- Institutional traders
- Retail traders

Trading vs. investing

Trading:

- Shorter holding period
- Focus on short-term trends or price fluctuations
- Take both long and short positions

Investing:

- Longer holding period
- Focus on market fundamentals
- Take mostly long positions

Financial time series data

Time series data: a sequence of data points indexed in time order

```
import pandas as pd
# Load the data
bitcoin_data = pd.read_csv("bitcoin_data.csv", index_col='Date', parse_dates=True)
print(bitcoin_data.head())
```

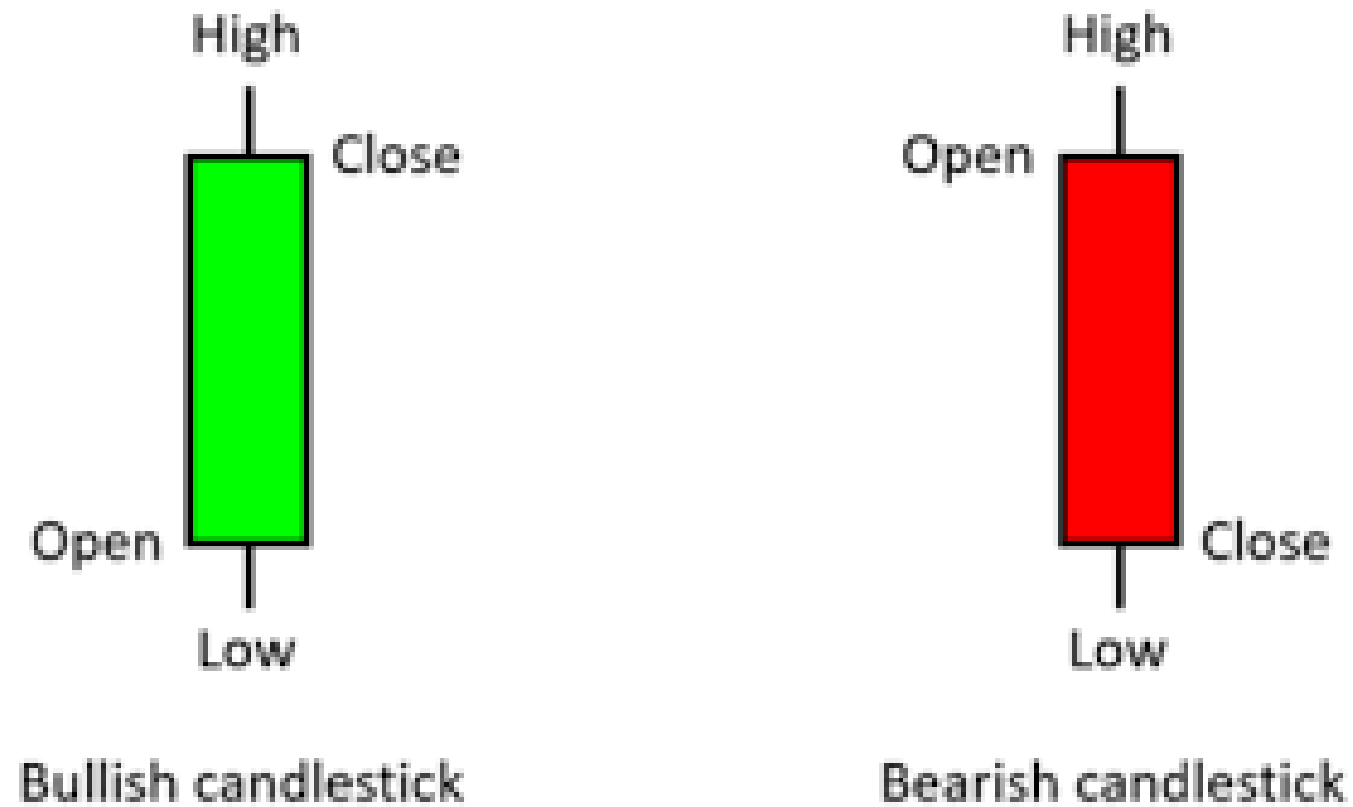
	Open	High	Low	Close	Volume
Date					
2019-01-01	3746.71	3850.91	3707.23	3843.52	4324200990
2019-01-02	3849.22	3947.98	3817.41	3943.41	5244856835
2019-01-03	3931.05	3935.69	3826.22	3836.74	4530215218
2019-01-04	3832.04	3865.93	3783.85	3857.72	4847965467
2019-01-05	3851.97	3904.90	3836.90	3845.19	5137609823

Plot line chart of time series data

```
import matplotlib.pyplot as plt
plt.plot(bitcoin_data['Close'], color='red')
plt.title("Daily close price")
plt.show()
```



Candlestick chart



- Each candlestick displays high, low, open, and close
- The color indicates bullish (rising prices) or bearish (falling prices) movement

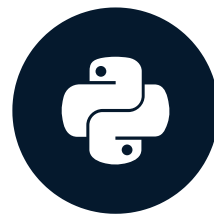
Plot candlestick chart with Python

```
import plotly.graph_objects as go
# Define the candlestick
candlestick = go.Candlestick(
    x = bitcoin_data.index,
    open = bitcoin_data['Open'],
    high = bitcoin_data['High'],
    low = bitcoin_data['Low'],
    close = bitcoin_data['Close'])
# Create a plot
fig = go.Figure(data=[candlestick])
# Show the plot
fig.show()
```



Getting familiar with your trading data

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

Different types of traders

- Day Trader: holds positions throughout the day but usually not overnight
- Swing Trader: holds positions from a few days to several weeks
- Position Trader: holds positions from a few months to several years

Resample the data

Date	Close
2019-11-29 04:00:00	1.1010
2019-11-29 08:00:00	1.1005
2019-11-29 12:00:00	1.0993
2019-11-29 16:00:00	1.1016
2019-11-29 20:00:00	1.1020

```
# Resample from hourly to daily
eurusd_daily = eurusd_h.resample('D').mean()
```

Date	Close
2019-11-25	1.10165
2019-11-26	1.10165
2019-11-27	1.10058
2019-11-28	1.10083
2019-11-29	1.10093

```
# Resample from hourly to weekly
eurusd_weekly = eurusd_h.resample('W').mean()
```

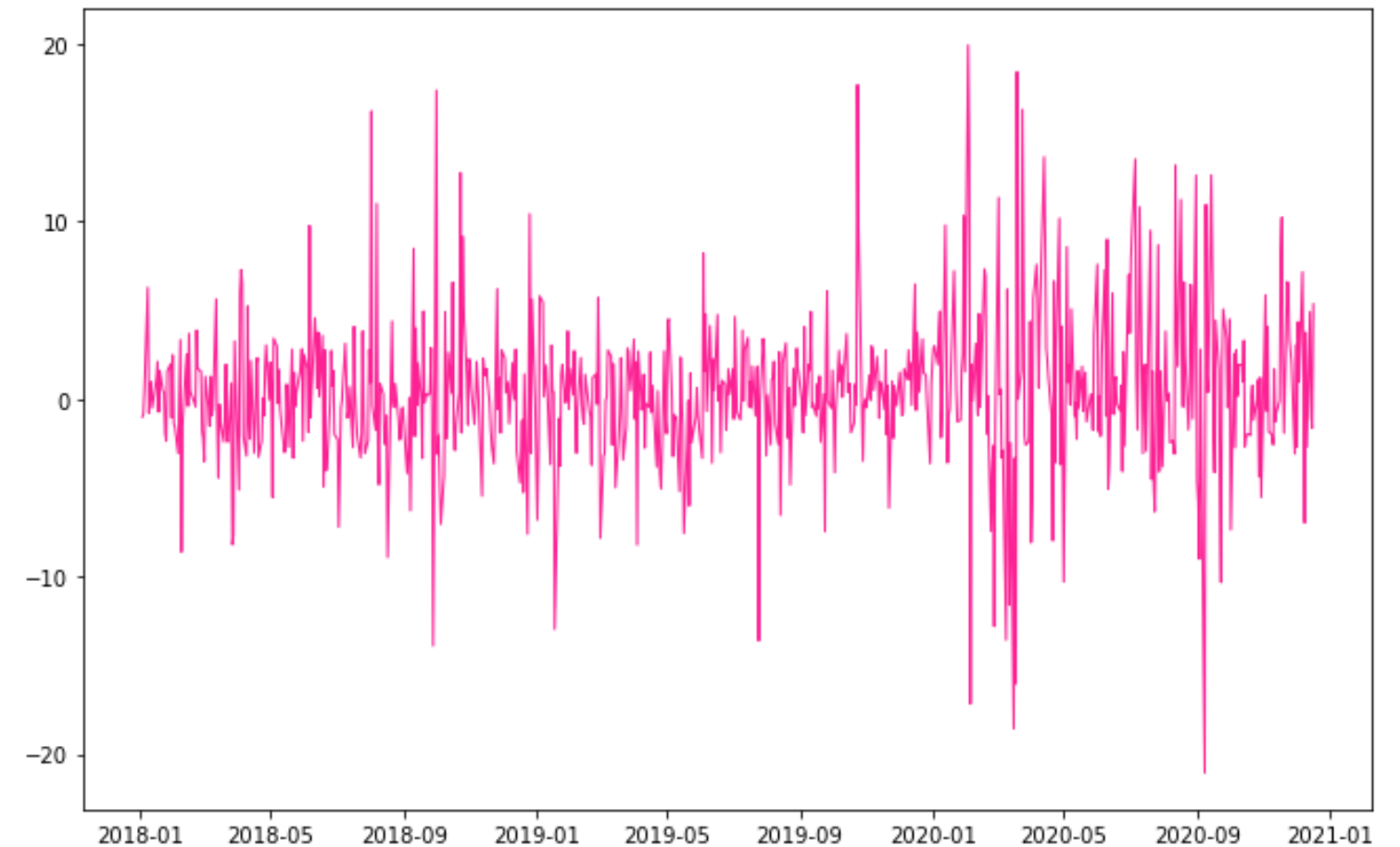
Date	Close
2019-11-03	1.11248
2019-11-10	1.10860
2019-11-17	1.10208
2019-11-24	1.10659
2019-12-01	1.10113

Calculate daily returns

```
# Calculate daily returns
stock_data['daily_return']
= stock_data['Close'].pct_change() * 100
```

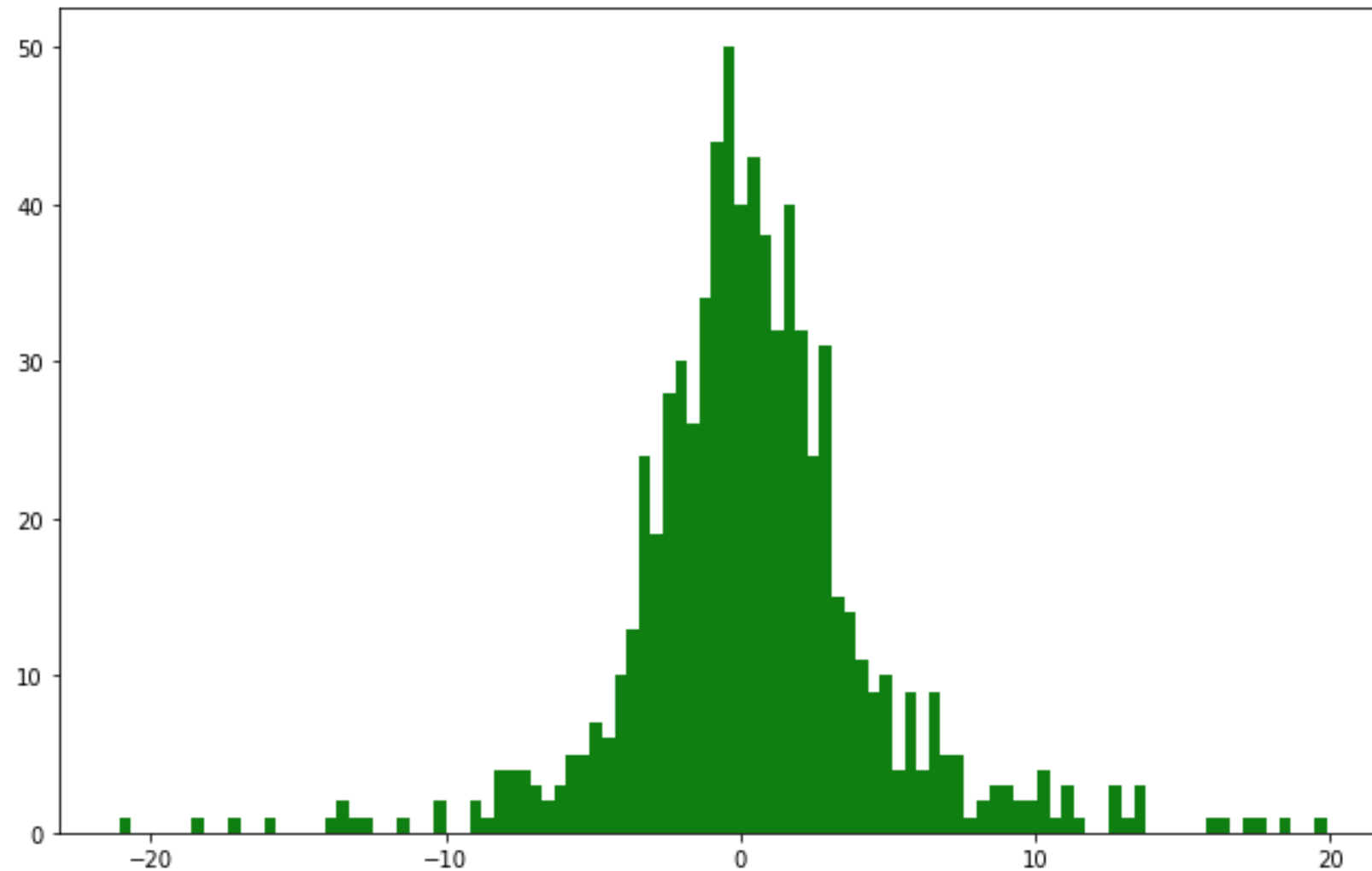
Date	Close	daily_return
2020-12-11	609.99	-2.723779
2020-12-14	639.83	4.891883
2020-12-15	633.25	-1.028398
2020-12-16	622.77	-1.654955
2020-12-17	655.90	5.319781

```
# Plot the data
plt.plot(stock_data['daily_return'])
plt.show()
```



Plot a histogram of daily returns

```
stock_data['daily_return'].hist(bins=100)  
plt.show()
```



Data transformation

Technical indicators: various types of data transformations

Simple moving average (SMA): arithmetic mean price over a specified n-period

```
stock_data['sma_50'] = stock_data['Close'].rolling(window=50).mean()
```

	Close	sma_50
Date		
2020-12-11	122.41	117.7474
2020-12-14	121.78	117.9226
2020-12-15	127.88	118.1502
2020-12-16	127.81	118.4432
2020-12-17	128.70	118.7156

Plot the rolling average

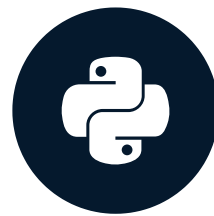
```
import matplotlib.pyplot as plt

plt.plot(stock_data['Close'],
         label='Close')
plt.plot(stock_data['sma_50'],
         label='SMA_50')
plt.legend()
plt.show()
```



Financial trading with bt

FINANCIAL TRADING IN PYTHON



Chelsea Yang
Data Science Instructor

The bt package

A flexible framework for defining and backtesting trading strategies

- Strategy: a method of buying and selling financial assets based on predefined rules
- Strategy backtesting: a way to assess the effectiveness of a strategy by testing it on historical data

```
import bt
```

The bt process

- Step 1: Get the historical price data
- Step 2: Define the strategy
- Step 3: Backtest the strategy with the data
- Step 4: Evaluate the result

Get the data

```
# Download historical prices
bt_data = bt.get('goog, amzn, tsla',
                 start='2020-6-1', end='2020-12-1')
print(bt_data.head())
```

	goog	amzn	tsla
Date			
2020-06-01	1431.819946	2471.040039	179.619995
2020-06-02	1439.219971	2472.409912	176.311996
2020-06-03	1436.380005	2478.399902	176.591995
2020-06-04	1412.180054	2460.600098	172.876007
2020-06-05	1438.390015	2483.000000	177.132004

Define the strategy

```
# Define the strategy
bt_strategy = bt.Strategy('Trade_Weekly',
                          [bt.algos.RunWeekly(), # Run weekly
                           bt.algos.SelectAll(), # Use all data
                           bt.algos.WeighEqually(), # Maintain equal weights
                           bt.algos.Rebalance()]) # Rebalance
```

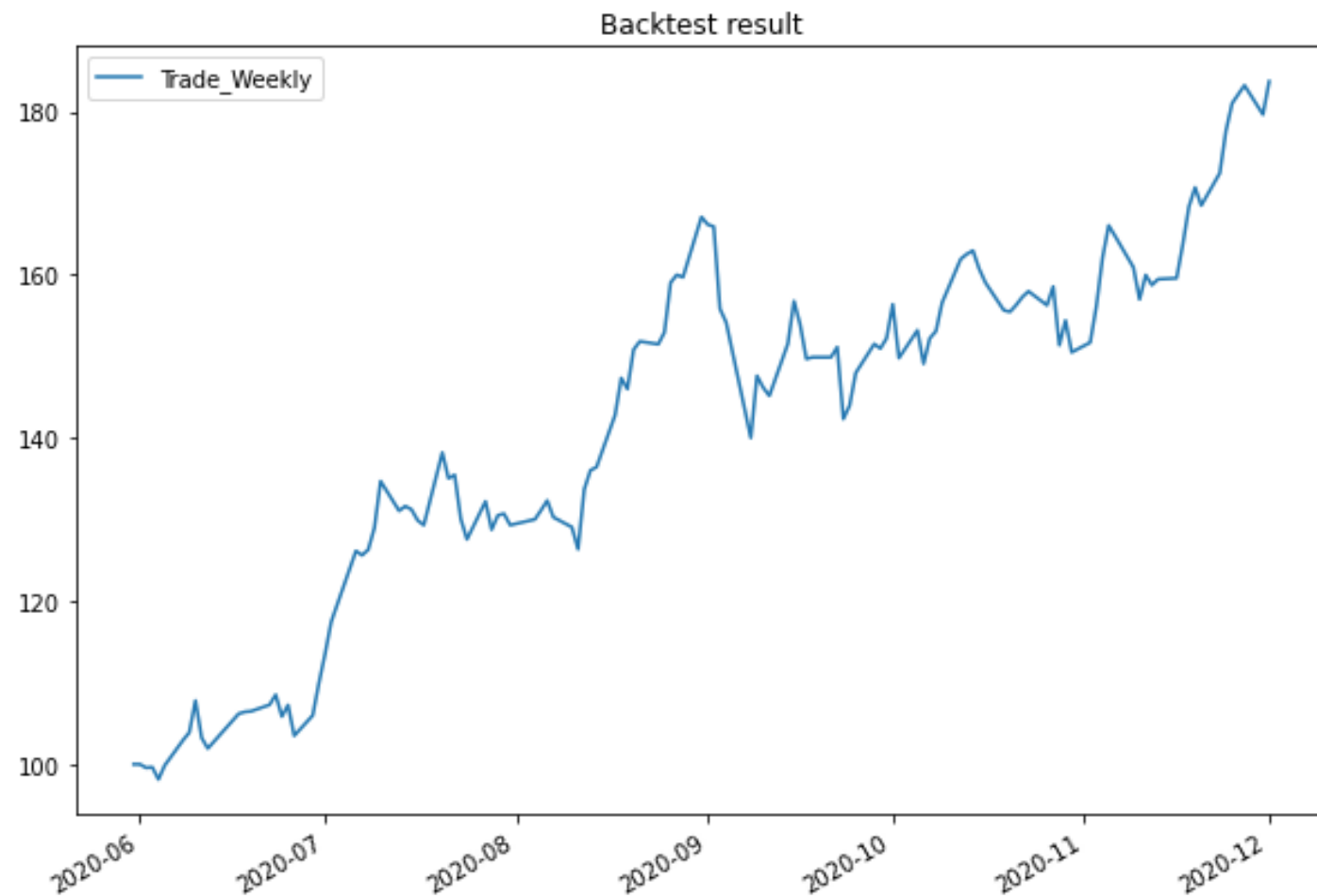
Backtest

```
# Create a backtest
bt_test = bt.Backtest(bt_strategy, bt_data)

# Run the backtest
bt_res = bt.run(bt_test)
```

Evaluate the result

```
# Plot the result
bt_res.plot(title="Backtest result")
```

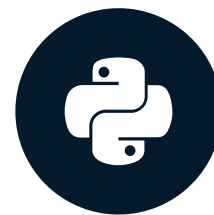


```
# Get trade details
bt_res.get_transactions()
```

Date	Security	price	quantity
2020-06-01	amaz	0.015000	22222222.0
	goog	1431.819946	232.0
	tsla	179.619995	1855.0
2020-06-08	amaz	0.011200	5700757.0
	goog	1446.609985	-16.0
...	...	...	...
2020-11-23	goog	1734.859985	23.0
	tsla	521.849976	-132.0
2020-11-30	amaz	0.004900	458015.0
	goog	1760.739990	7.0
	tsla	567.599976	-27.0

Trend indicator MAs

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

What are technical indicators?

- Mathematical calculations based on historical market data
- Assume the market is efficient and the price has incorporated all public information
- Help traders to gain insight into past price patterns

Types of indicators

- **Trend indicators:** measure the direction or strength of a trend
 - Example: Moving averages (MA), Average Directional Movement Index (ADX)
- **Momentum indicators:** measure the velocity of price movement
 - Example: Relative Strength Index (RSI)
- **Volatility indicators:** measure the magnitude of price deviations
 - Example: Bollinger Bands

The TA-Lib package

TA-Lib : Technical Analysis Library

- Includes 150+ technical indicator implementations

```
import talib
```

Moving average indicators

- SMA: Simple Moving average
- EMA: Exponential Moving Average
- Characteristics:
 - Move with the price
 - Smooth out the data to better indicate the price direction

Simple Moving Average (SMA)

$$SMA = (P_1 + P_2 + \dots + P_n) / n$$

```
# Calculate two SMAs
stock_data['SMA_short'] = talib.SMA(stock_data['Close'], timeperiod=10)
stock_data['SMA_long'] = talib.SMA(stock_data['Close'], timeperiod=50)
# Print the last five rows
print(stock_data.tail())
```

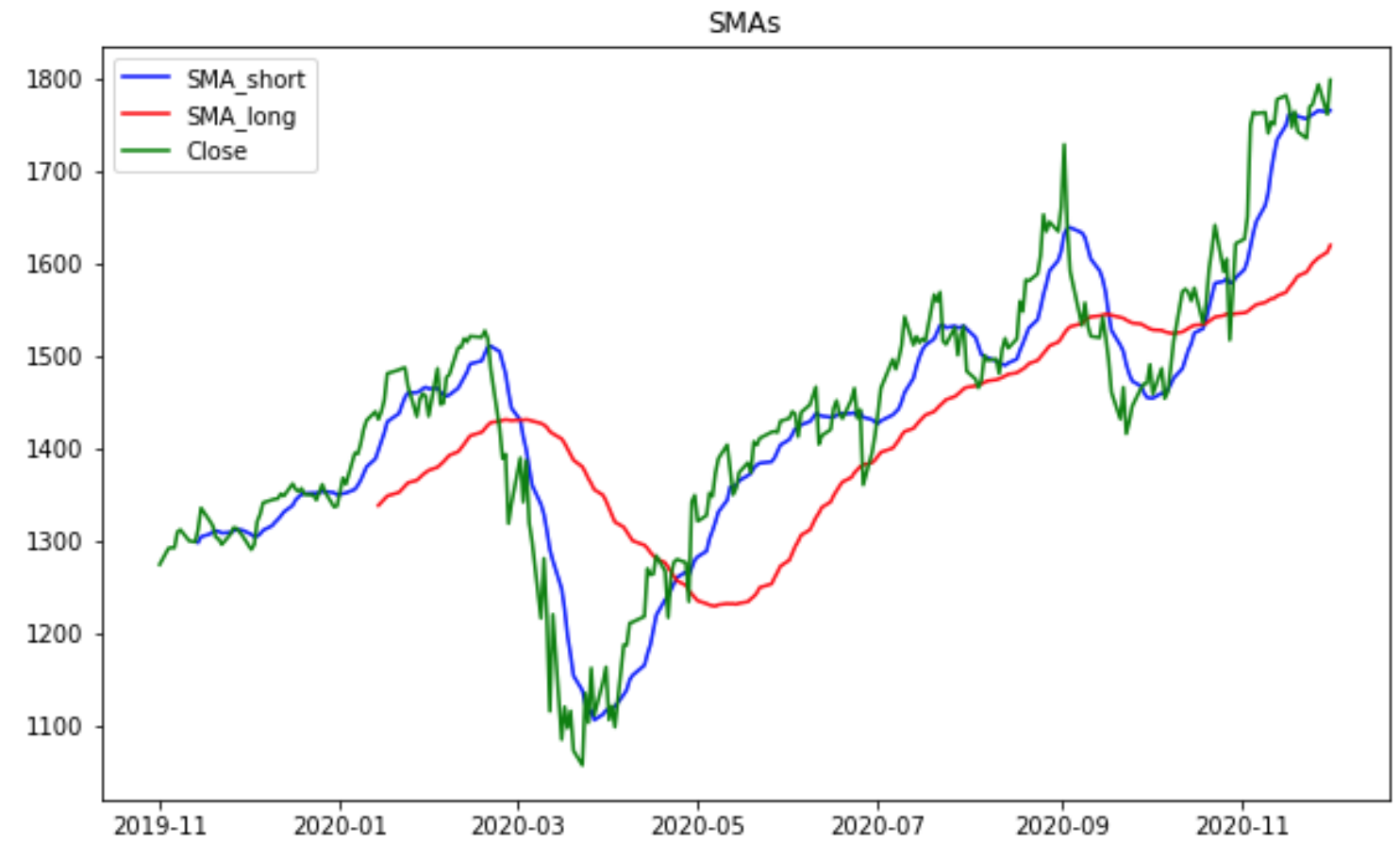
	Close	SMA_short	SMA_long
Date			
2020-11-24	1768.88	1758.77	1594.74
2020-11-25	1771.43	1760.65	1599.75
2020-11-27	1793.19	1764.98	1605.70
2020-11-30	1760.74	1763.35	1611.71
2020-12-01	1798.10	1765.02	1619.05

Plotting the SMA

```
import matplotlib.pyplot as plt

# Plot SMA with the price
plt.plot(stock_data['SMA_short'],
         label='SMA_short')
plt.plot(stock_data['SMA_long'],
         label='SMA_long')
plt.plot(stock_data['Close'],
         label='Close')

# Customize and show the plot
plt.legend()
plt.title('SMAs')
plt.show()
```



Exponential Moving Average (EMA)

$$EMA_n = P_n \times multiplier + \text{previous EMA} \times (1 - multiplier)$$

$$multiplier = 2/(n + 1)$$

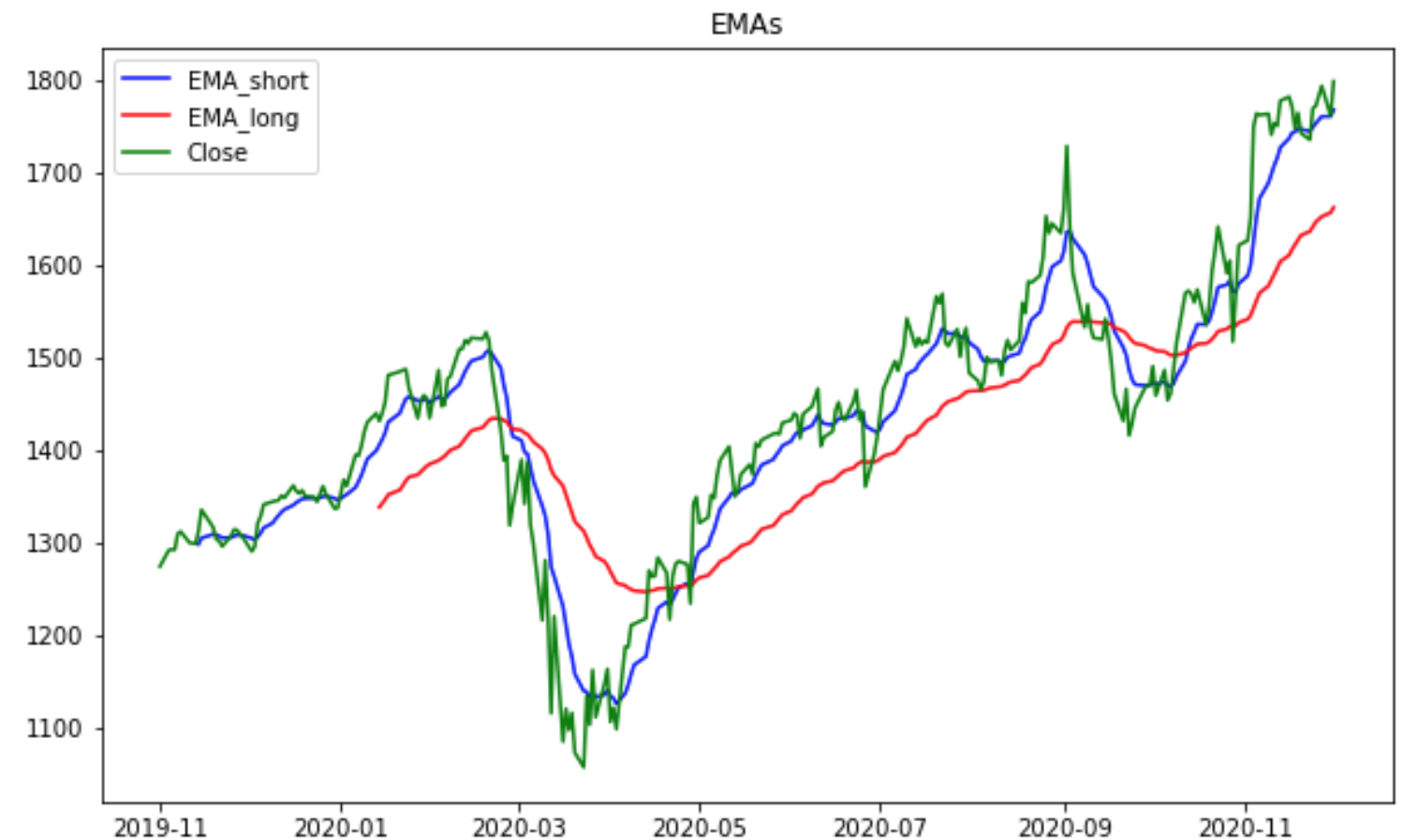
```
# Calculate two EMAs
stock_data['EMA_short'] = talib.EMA(stock_data['Close'], timeperiod=10)
stock_data['EMA_long'] = talib.EMA(stock_data['Close'], timeperiod=50)
# Print the last five rows
print(stock_data.tail())
```

	Close	EMA_short	EMA_long
Date			
2020-11-24	1768.88	1748.65	1640.98
2020-11-25	1771.43	1752.79	1646.09
2020-11-27	1793.19	1760.13	1651.86
2020-11-30	1760.74	1760.24	1656.13
2020-12-01	1798.10	1767.13	1661.70

Plotting the EMA

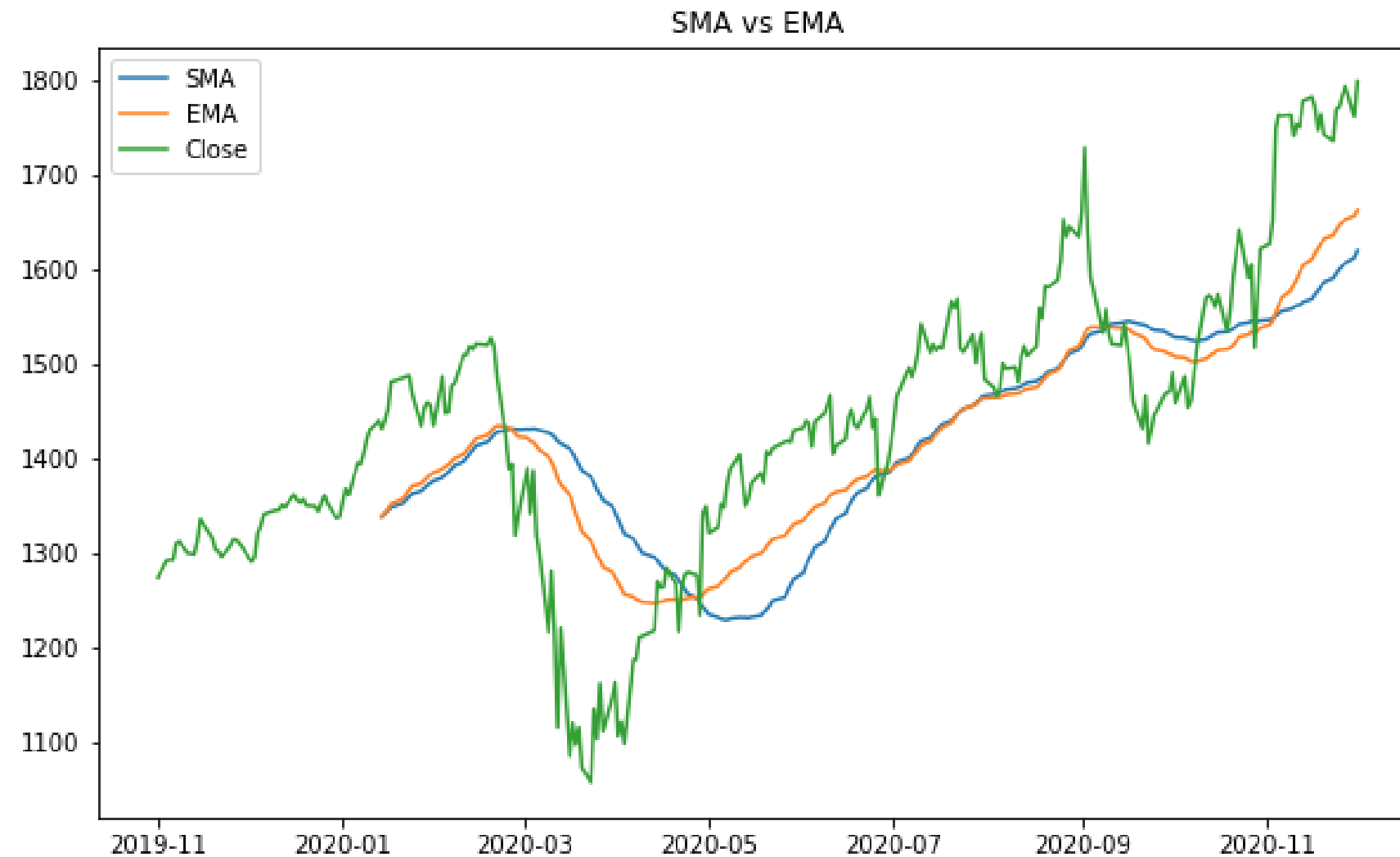
```
import matplotlib.pyplot as plt
# Plot EMA with the price
plt.plot(stock_data['EMA_short'],
         label='EMA_short')
plt.plot(stock_data['EMA_long'],
         label='EMA_long')
plt.plot(stock_data['Close'],
         label='Close')

# Customize and show the plot
plt.legend()
plt.title('EMAs')
plt.show()
```



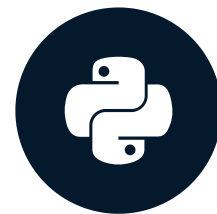
SMA vs. EMA

EMA is more sensitive to the most recent price movement



Strength indicator: ADX

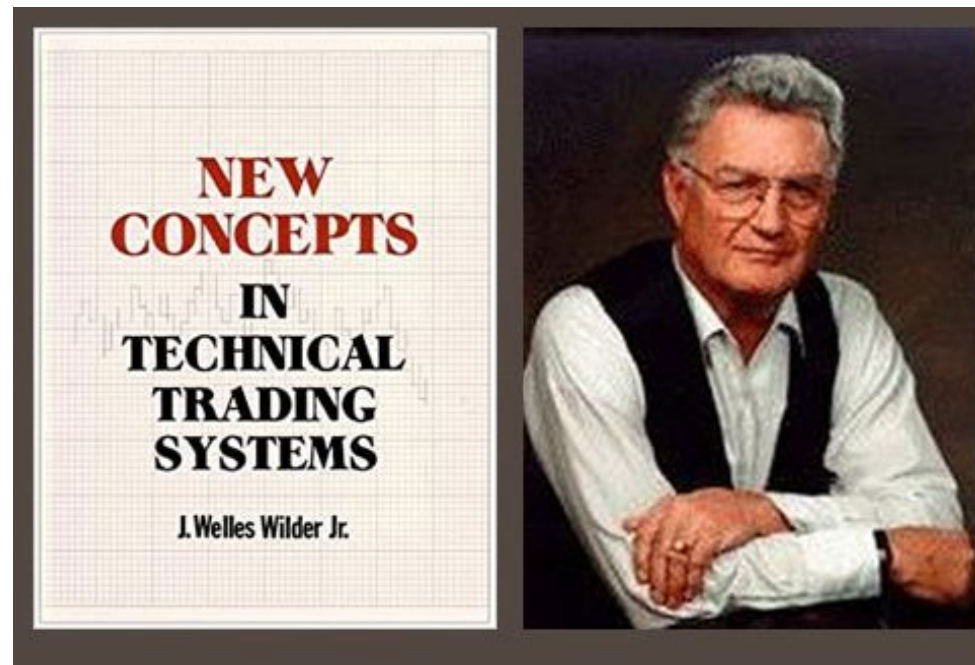
FINANCIAL TRADING IN PYTHON



Chelsea Yang
Data Science Instructor

What is ADX?

- Stands for "Average Directional Movement Index"
- Developed by J. Welles Wilder
 - "New Concepts in Technical Systems" (1987)
- Measures the strength of a trend
 - Oscillates between 0 and 100
 - $ADX \leq 25$: no trend
 - $ADX > 25$: trending market
 - $ADX > 50$: strong trending market



How is ADX calculated?

- Derived from the smoothed averages of the difference between +DI and -DI
 - +DI (Plus Directional Index): quantify the presence of an uptrend
 - -DI (Minus Directional Index): quantify the presence of a downtrend
- Calculation input:
 - high, low, and close prices of each period

Implementing ADX in Python

```
# Calculate ADX
stock_data['ADX'] = talib.ADX(stock_data['High'], stock_data['Low'], stock_data['Close'],
                             timeperiod=14)

# Print the last five rows
print(stock_data.tail())
```

	Open	High	Low	Close	ADX
Date					
2020-11-24	540.40	559.99	526.20	555.38	21.16
2020-11-25	550.06	574.00	545.37	574.00	23.92
2020-11-27	581.16	598.78	578.45	585.76	26.82
2020-11-30	602.21	607.80	554.51	567.60	28.25
2020-12-01	597.59	597.85	572.05	584.76	29.58

Plotting ADX

```
import matplotlib.pyplot as plt

# Create subplots
fig, (ax1, ax2) = plt.subplots(2)

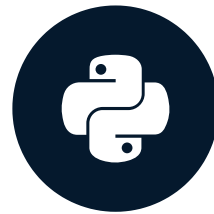
# Plot ADX with the price
ax1.set_ylabel('Price')
ax1.plot(stock_data['Close'])
ax2.set_ylabel('ADX')
ax2.plot(stock_data['ADX'])

ax1.set_title('Price and ADX')
plt.show()
```



Momentum indicator: RSI

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

What is RSI?

- Stands for "Relative Strength Index"
- Developed by J. Welles Wilder
 - "New Concepts in Technical Systems" (1987)
- Measures the momentum of a trend
 - Oscillates between 0 and 100
 - $RSI > 70$: Overbought
 - $RSI < 30$: Oversold

How is RSI calculated?

$$RSI = 100 - 100 / (1 + RS)$$

Where:

- RS: relative strength
- RS = average of upward price changes / average of downward price changes

Implementing RSI in Python

```
# Calculate RSI
stock_data['RSI'] = talib.RSI(stock_data['Close'], timeperiod=14)
# Print the last five rows
print(stock_data.tail())
```

	Open	High	Low	Close	RSI
Date					
2020-11-24	1730.50	1771.60	1727.69	1768.88	62.78
2020-11-25	1772.89	1778.54	1756.54	1771.43	63.10
2020-11-27	1773.09	1804.00	1772.44	1793.19	65.81
2020-11-30	1781.18	1788.06	1755.00	1760.74	58.87
2020-12-01	1774.37	1824.83	1769.37	1798.10	63.63

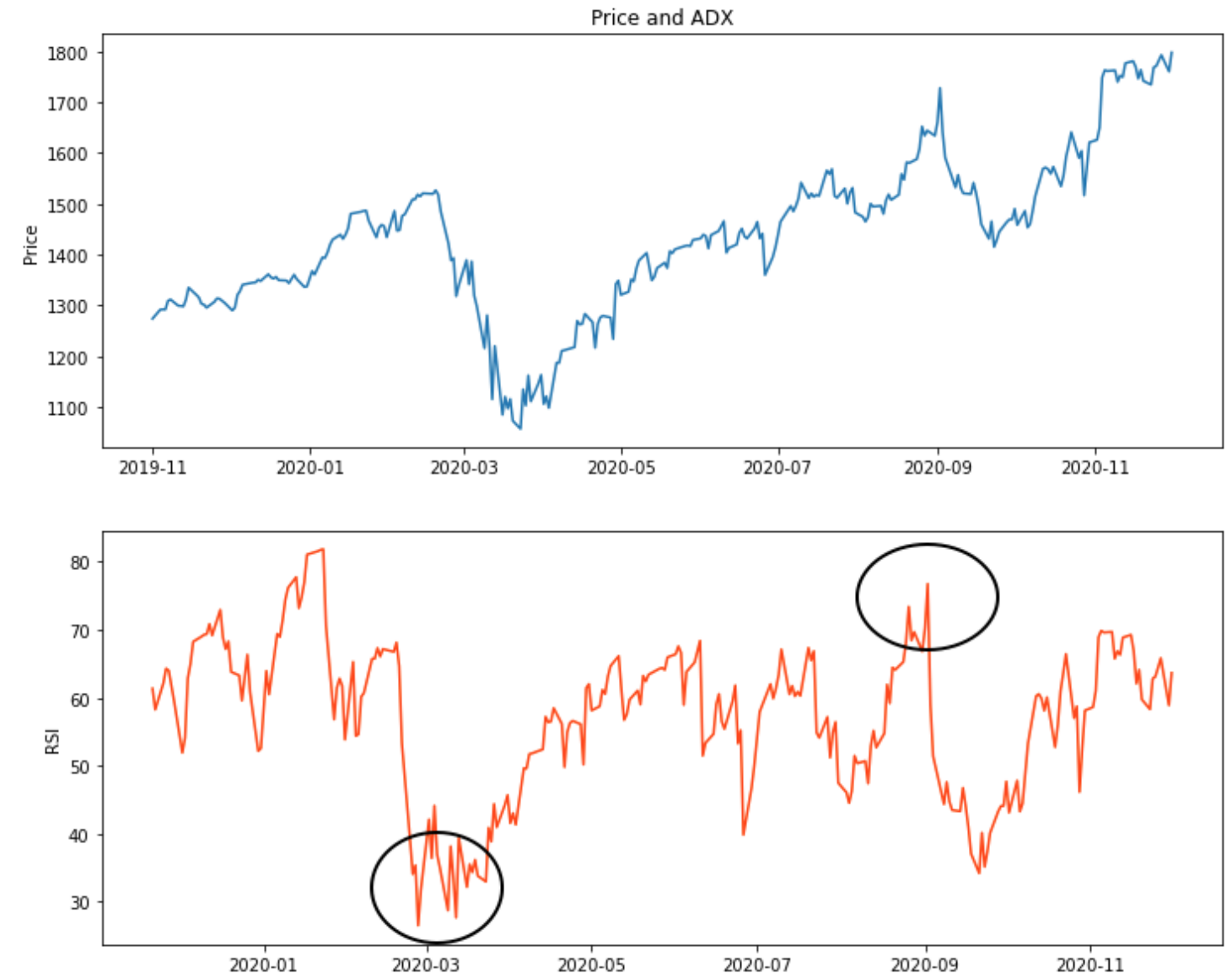
Plotting RSI

```
import matplotlib.pyplot as plt

# Create subplots
fig, (ax1, ax2) = plt.subplots(2)

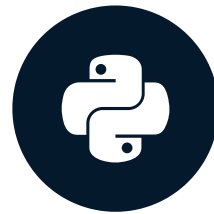
# Plot RSI with the price
ax1.set_ylabel('Price')
ax1.plot(stock_data['Close'])
ax2.set_ylabel('RSI')
ax2.plot(stock_data['RSI'])

ax1.set_title('Price and RSI')
plt.show()
```



Volatility indicator: Bollinger Bands

FINANCIAL TRADING IN PYTHON

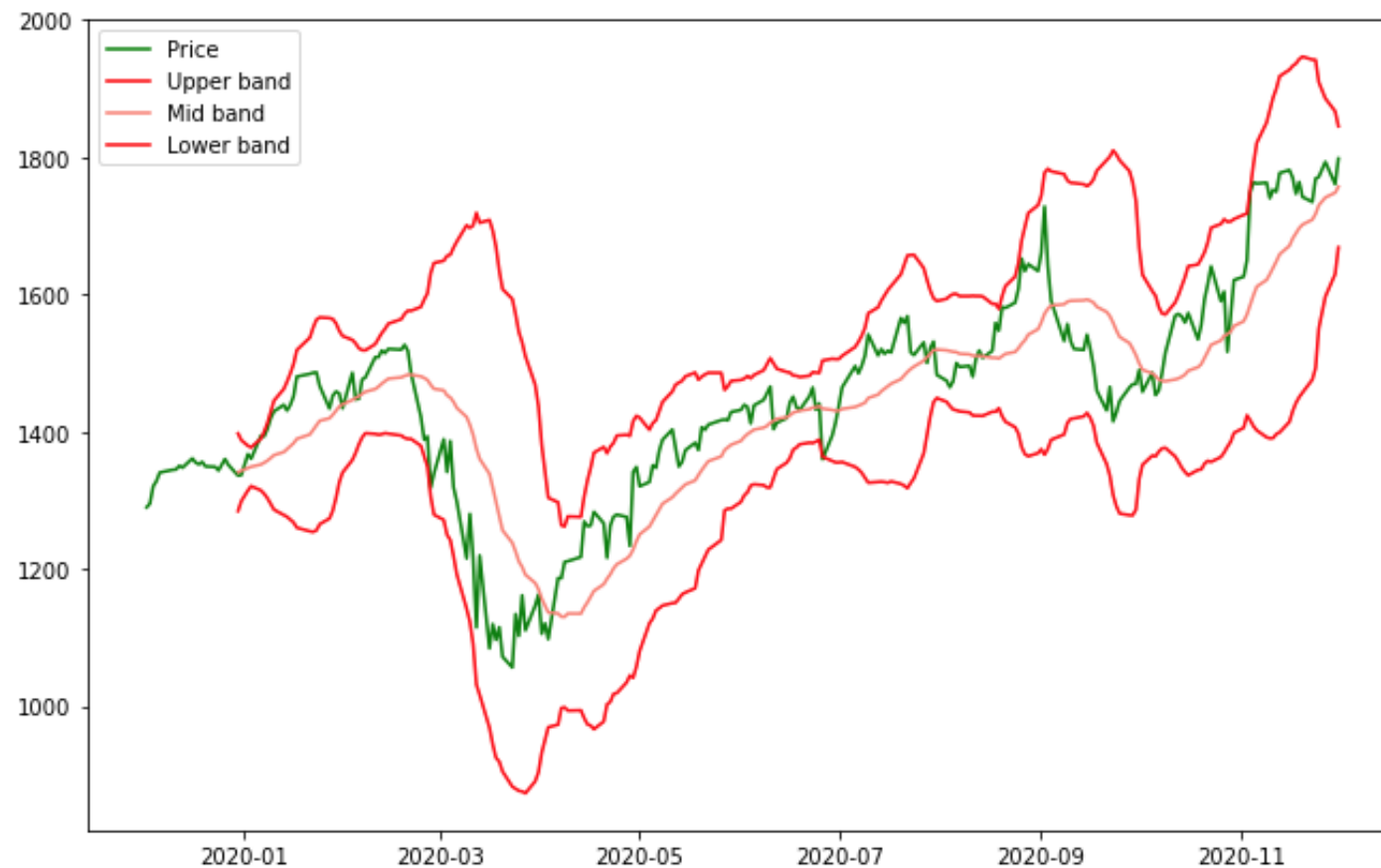


Chelsea Yang

Data Science Instructor

What are Bollinger Bands?

- Developed by John Bollinger
 - "Bollinger on Bollinger Bands"



- Measure price volatility
- Composed of three lines:
 - Middle band: n-period simple moving average
 - Upper band: k-standard deviations above the middle band
 - Lower band: k-standard deviations below the middle band

Bollinger Bands implications

- The wider the bands, the more volatile the asset prices.
- Measure whether a price is too high or too low on a relative basis:
 - Relatively high: price close to the upper band
 - Relatively low: price close to the lower band

Implementing Bollinger Bands in Python

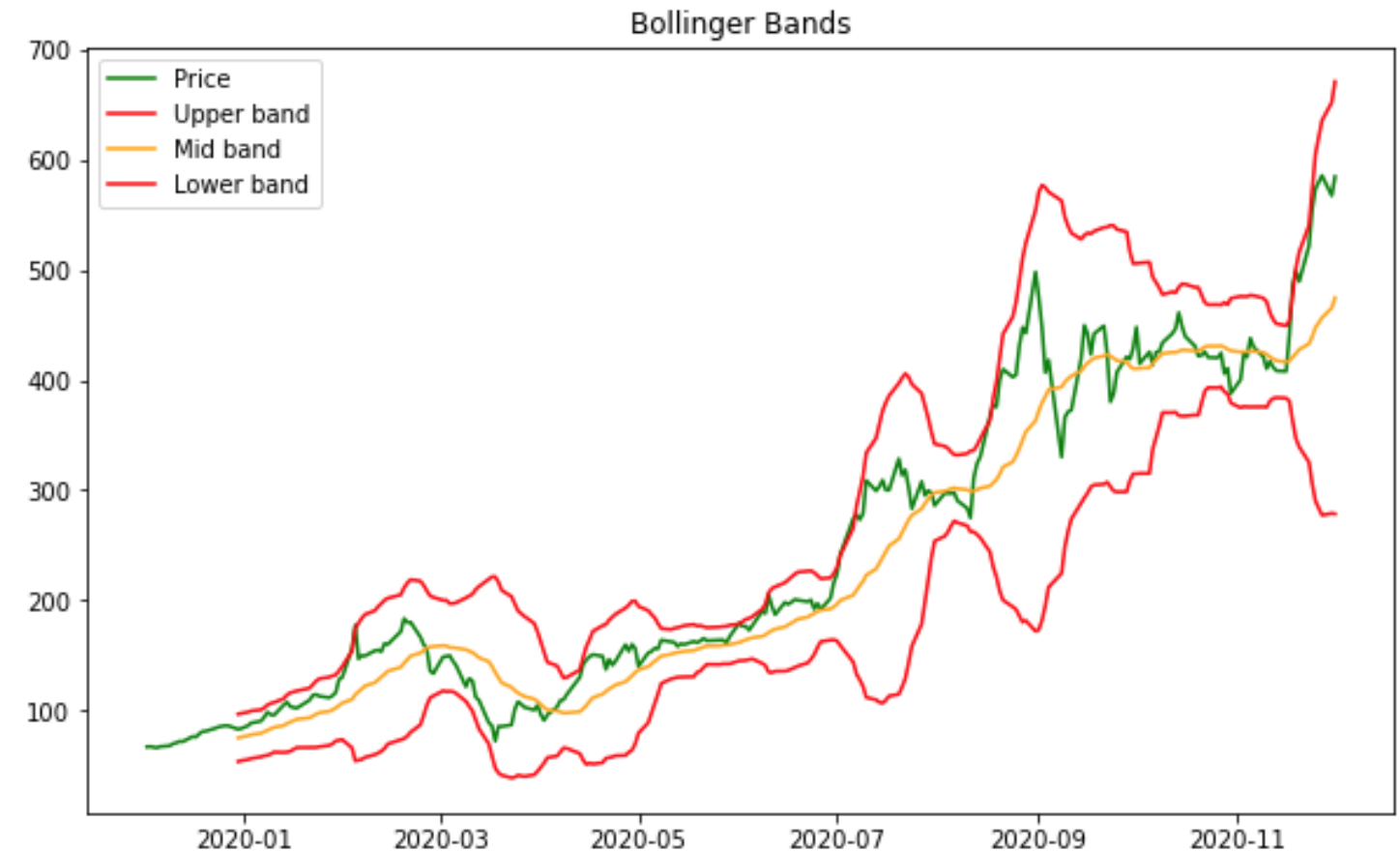
```
# Define the Bollinger Bands
upper, mid, lower = talib.BBANDS(stock_data['Close'],
                                nbdevup=2,
                                nbdevdn=2,
                                timeperiod=20)
```

Plotting Bollinger Bands

```
import matplotlib.pyplot as plt

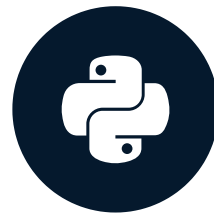
# Plot the Bollinger Bands
plt.plot(stock_data['Close'], label='Price')
plt.plot(upper, label="Upper band")
plt.plot(mid, label='Middle band')
plt.plot(lower, label='Lower band')

# Customize and show the plot
plt.title('Bollinger Bands')
plt.legend()
plt.show()
```



Trading signals

FINANCIAL TRADING IN PYTHON



Chelsea Yang

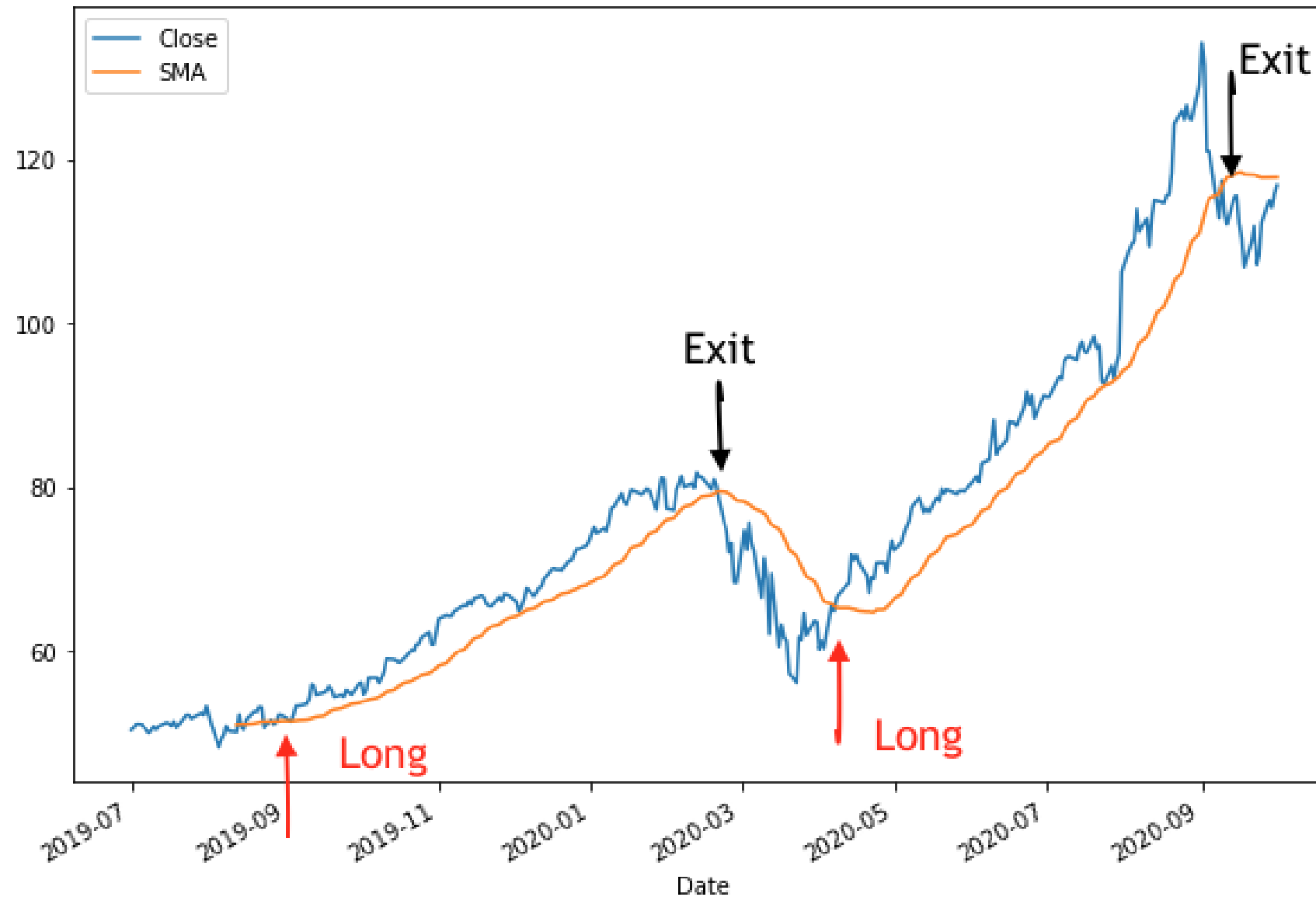
Data Science Instructor

What are trading signals?

- Triggers to long or short financial assets based on predetermined criteria
- Can be constructed using:
 - One technical indicator
 - Multiple technical indicators
 - A combination of market data and indicators
- Commonly used in algorithmic trading

A signal example

- Signal: Price > SMA (long when the price rises above the SMA)



How to implement signals in bt

1. Get the data and calculate indicators
2. Define the signal-based strategy
 - `bt.algos.SelectWhere()`
 - `bt.algos.WeighTarget()`
3. Create and run a backtest
4. Review the backtest result

Construct the signal

```
# Get price data by the stock ticker
price_data = bt.get('aapl', start='2019-11-1', end='2020-12-1')
```

```
# Calculate SMA
sma = price_data.rolling(20).mean()
```

Alternatively, use `talib` to calculate the indicator:

```
# Calculate SMA
import talib
sma = talib.SMA(price_data['Close'], timeperiod=20)
```

Define a signal-based strategy

```
# Define the signal-based strategy
bt_strategy = bt.Strategy('AboveEMA',
                          [bt.algos.SelectWhere(price_data > sma),
                           bt.algos.WeighEqually(),
                           bt.algos.Rebalance()]])
```

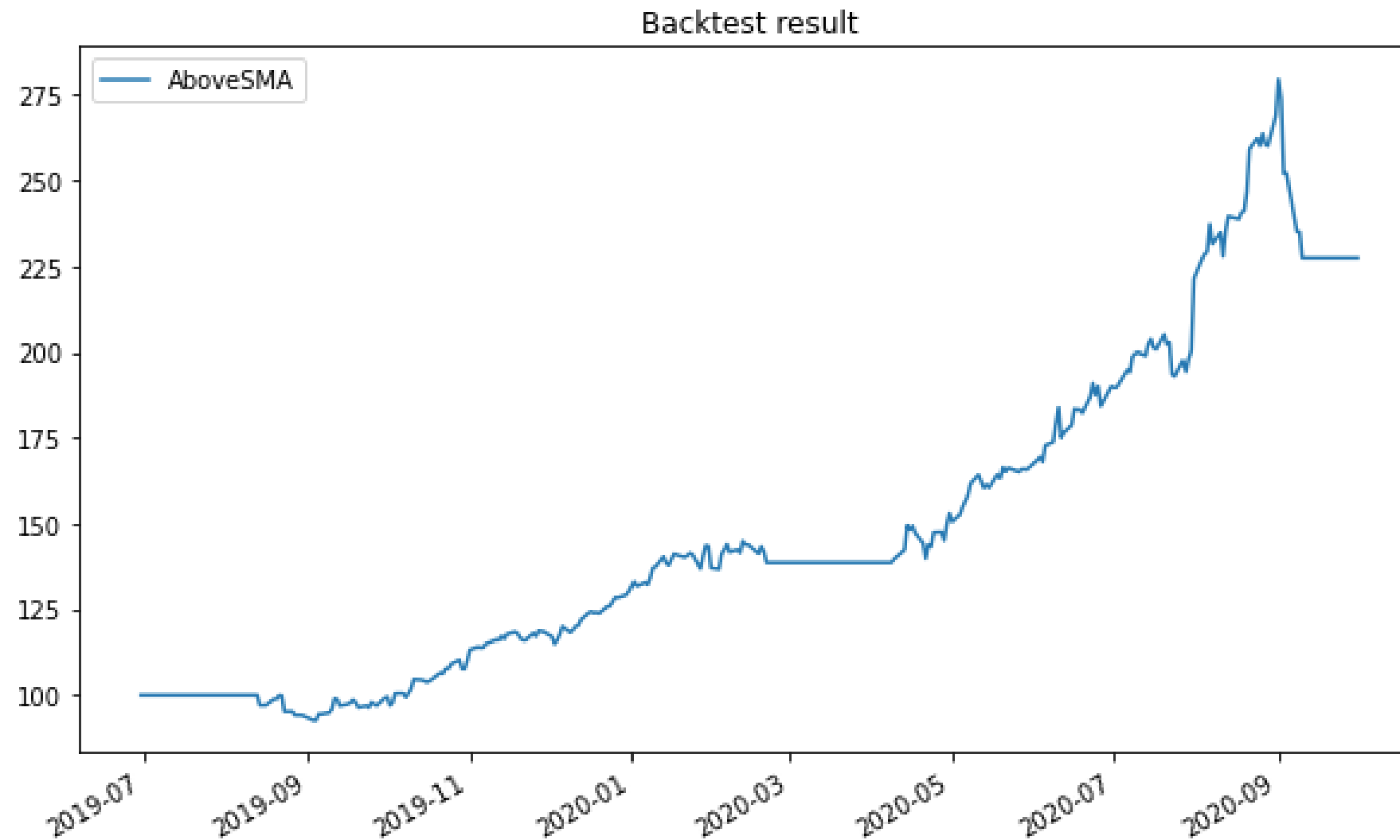
- For simplicity, we assume:
 - Trade one asset at a time
 - No slippage or commissions
 - Slippage: the difference between the expected price of a trade and the price at which the trade is executed
 - Commission: fees charged by brokers when executing a trade

Backtest the signal based strategy

```
# Create the backtest and run it
bt_backtest = bt.Backtest(bt_strategy, price_data)
bt_result = bt.run(bt_backtest)
```

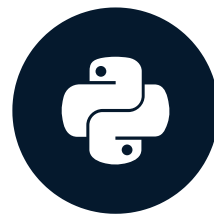
Plot the backtest result

```
# Plot the backtest result  
bt_result.plot(title='Backtest result')
```



Trend-following strategies

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

Two types of trading strategies

Trend-following

- Bet the price trend will continue in the same direction
- Use trend indicators such as moving averages, ADX, etc to construct trading signals

Mean reversion

- Bet the price tends to reverse back towards the mean
- Use indicators such as RSI, Bollinger Bands, etc, to construct trading signals

MA crossover strategy

The trend is your friend.

- Two EMA crossover:
 - Long signal: the short-term EMA crosses above the long-term EMA
 - Short signal: the short-term EMA crosses below the long-term EMA

Calculate the indicators

```
import talib
# Calculate the indicators
EMA_short = talib.EMA(price_data['Close'],
                      timeperiod=10).to_frame()
EMA_long = talib.EMA(price_data['Close'],
                     timeperiod=40).to_frame()
```

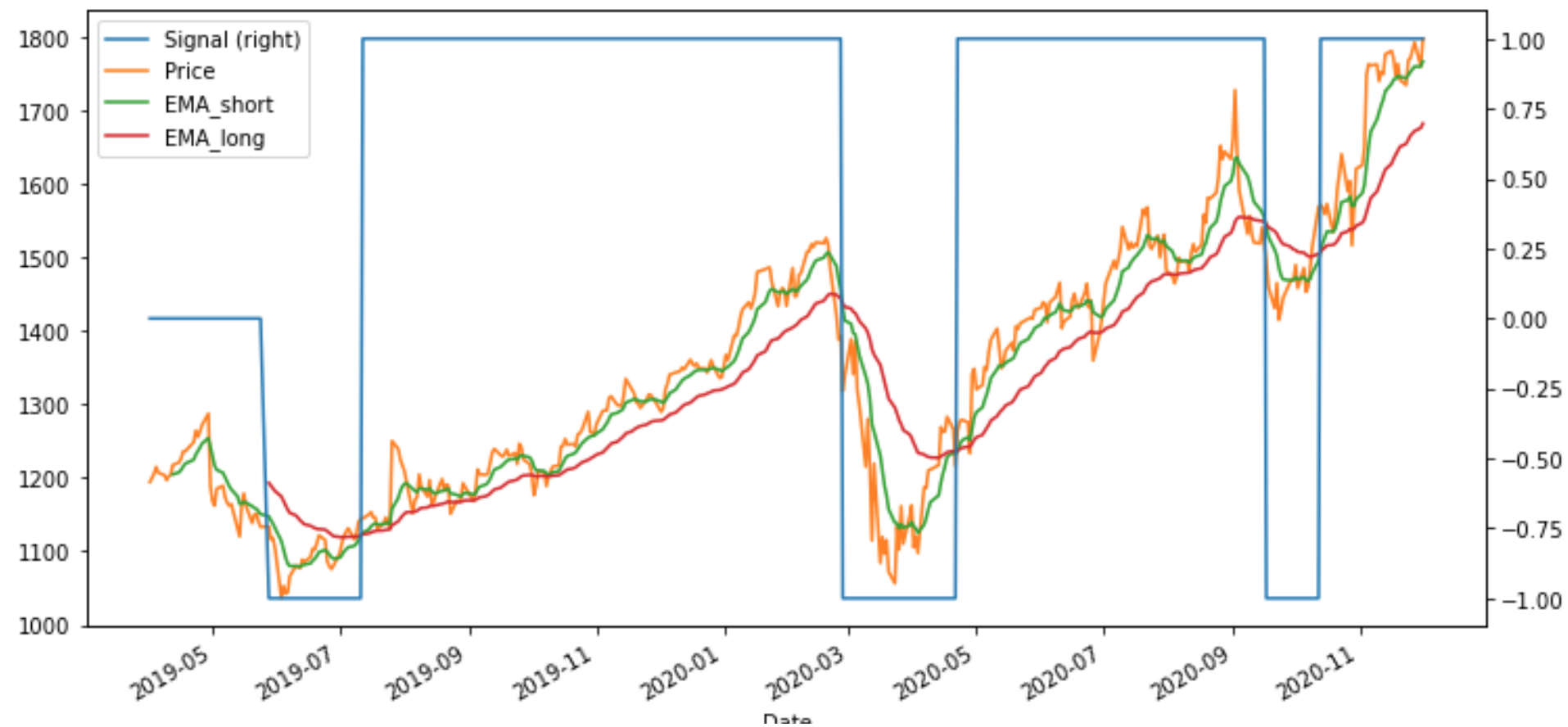
Construct the signal

```
# Create the signal DataFrame
signal = EMA_long.copy()
signal[EMA_long.isnull()] = 0

# Construct the signal
signal[EMA_short > EMA_long] = 1
signal[EMA_short < EMA_long] = -1
```

Plot the signal

```
# Plot the signal, price and MAs
combined_df = bt.merge(signal, price_data, EMA_short, EMA_long)
combined_df.columns = ['Signal', 'Price', 'EMA_short', 'EMA_long']
combined_df.plot(secondary_y=['Signal'])
```



Define the strategy with the signal

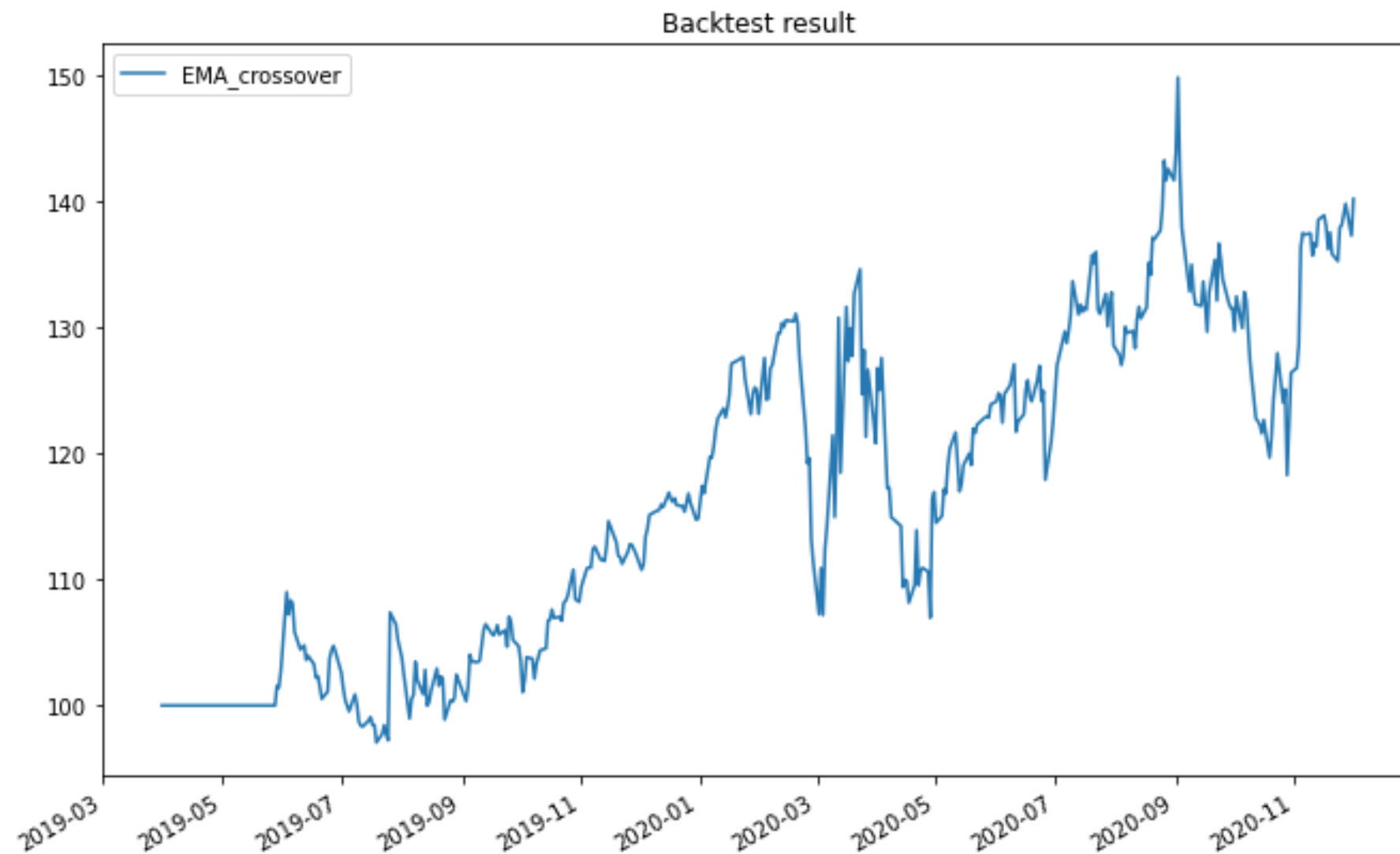
```
# Define the strategy
bt_strategy = bt.Strategy('EMA_crossover',
                          [bt.algos.WeighTarget(signal),
                           bt.algos.Rebalance()])
```

Backtest the signal based strategy

```
# Create the backtest and run it
bt_backtest = bt.Backtest(bt_strategy, price_data)
bt_result = bt.run(bt_backtest)
```

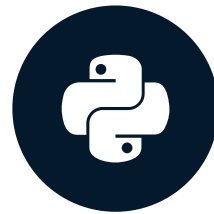

Plot backtest results

```
# Plot the backtest result  
bt_result.plot(title='Backtest result')
```



Mean reversion strategy

FINANCIAL TRADING IN PYTHON



Chelsea Yang
Data Science Instructor

RSI-based mean reversion strategy

Buy the fear and sell the greed

- RSI-based mean reversion strategy:
 - Short signal: $RSI > 70$
 - Suggests the asset is likely overbought and the price may soon reverse
 - Long signal: $RSI < 30$
 - Suggests the asset is likely oversold and the price may soon rally

Calculate the indicator

```
import talib
# Calculate the RSI
stock_rsi = talib.RSI(price_data['Close']).to_frame()
```

Construct the signal

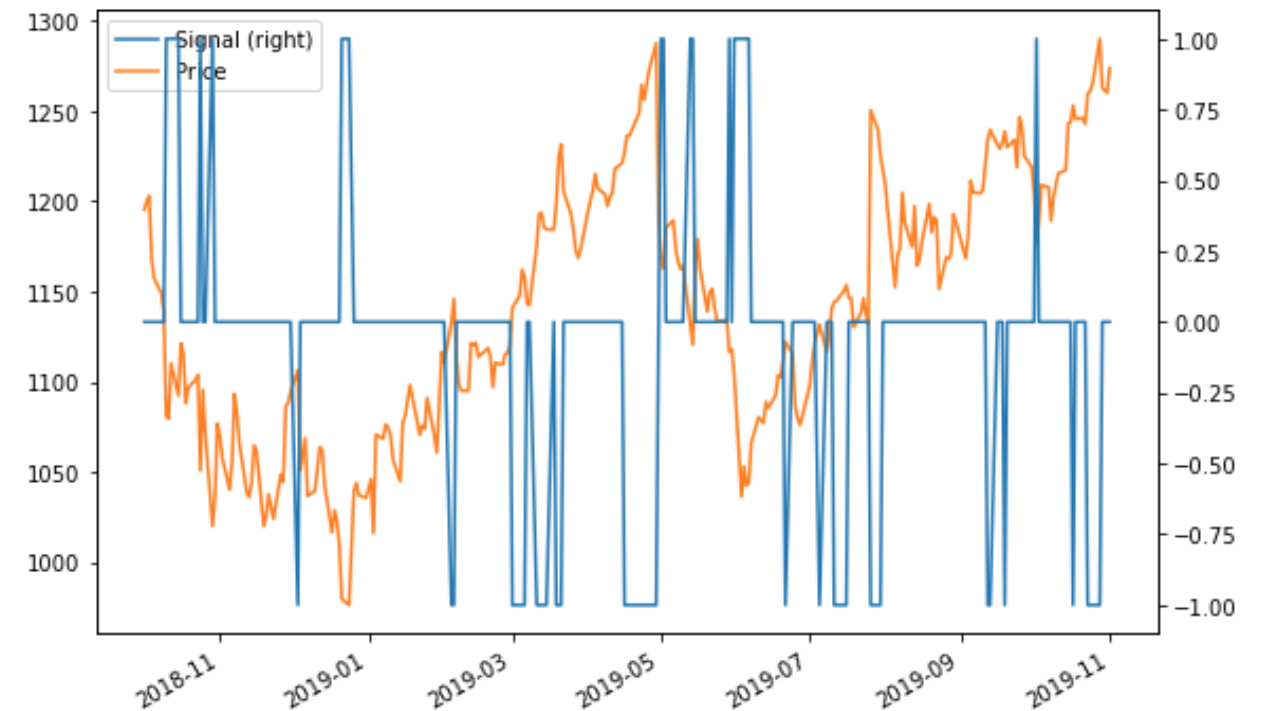
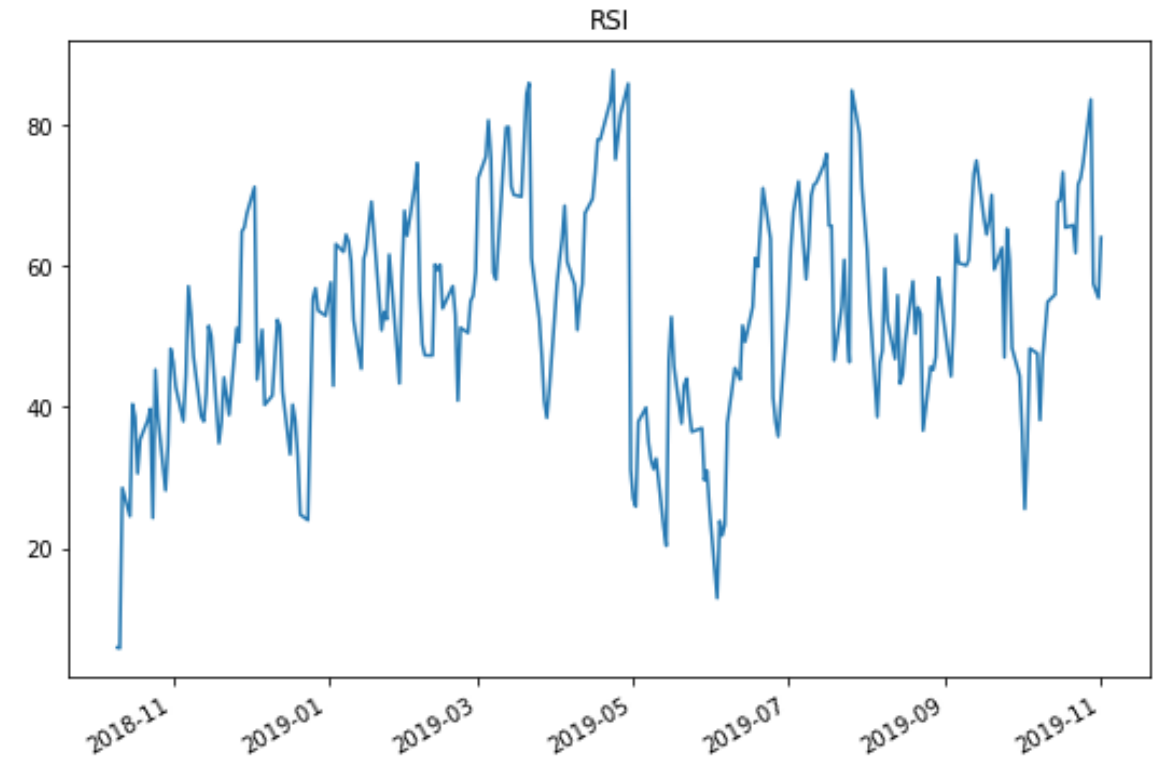
```
# Create the same DataFrame structure as RSI
signal = stock_rsi.copy()
signal[stock_rsi.isnull()] = 0

# Construct the signal
signal[stock_rsi < 30] = 1
signal[stock_rsi > 70] = -1
signal[(stock_rsi <= 70) & (stock_rsi >= 30)] = 0
```

Plot the signal

```
# Plot the RSI
stock_rsi.plot()
plt.title('RSI')
```

```
# Merge data into one DataFrame
combined_df = bt.merge(signal, stock_data)
combined_df.columns = ['Signal', 'Price']
# Plot the signal with price
combined_df.plot(secondary_y = ['Signal'])
```



Define the strategy with the signal

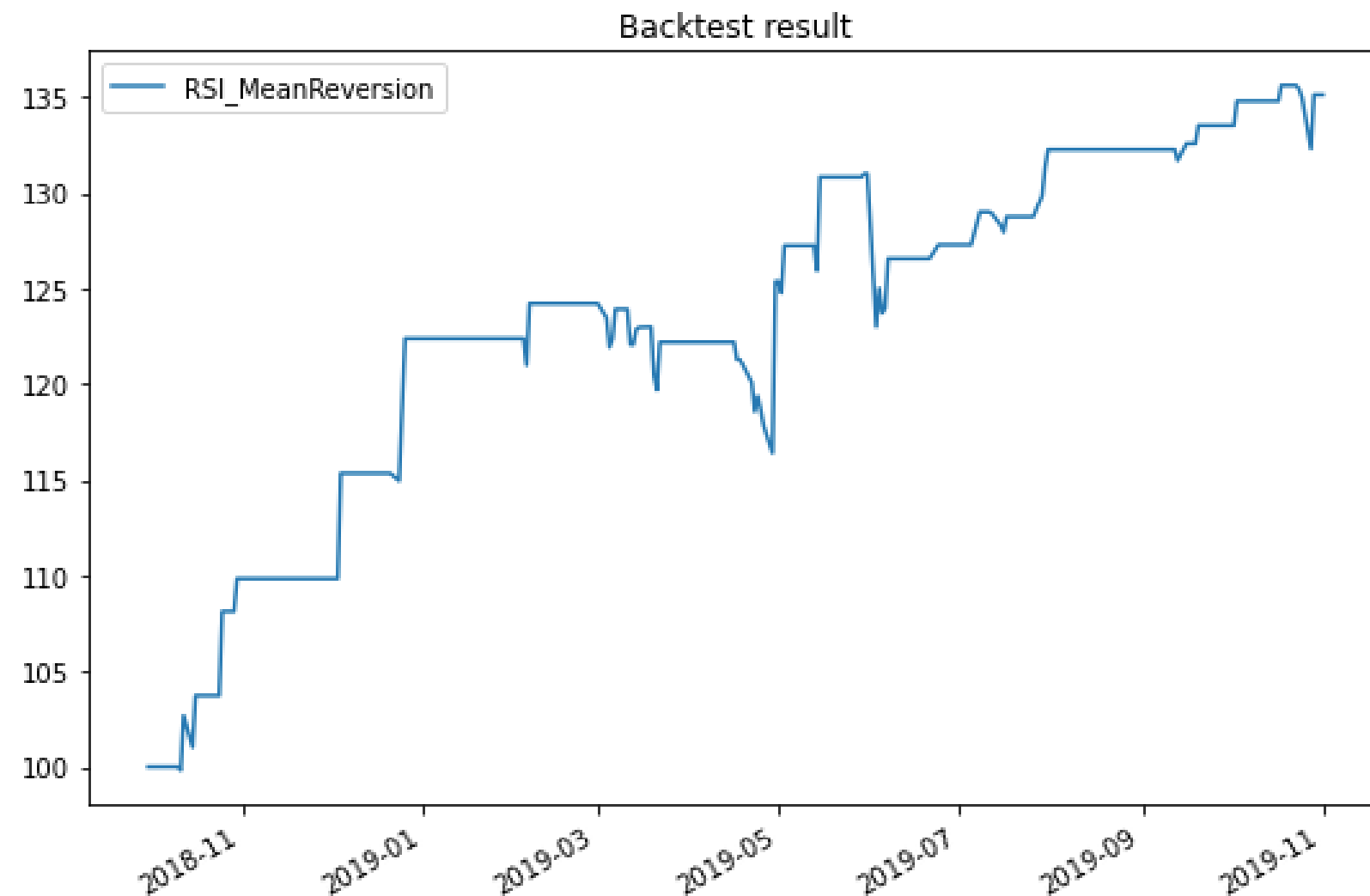
```
# Define the strategy
bt_strategy = bt.Strategy('RSI_MeanReversion',
                          [bt.algos.WeighTarget(signal),
                           bt.algos.Rebalance()])
```

Backtest the signal-based strategy

```
# Create the backtest and run it  
bt_backtest = bt.Backtest(bt_strategy, price_data)  
bt_result = bt.run(bt_backtest)
```

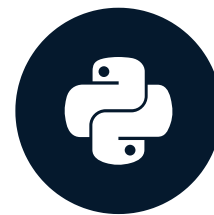

Plot backtest result

```
# Plot the backtest result  
bt_result.plot(title='Backtest result')
```



Strategy optimization and benchmarking

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

How to decide the values of input parameters?

- Question:
 - Which is a better SMA lookback period to use in the signal "Price > SMA"?
- Solution: strategy optimization
 - Try a range of input parameter values in backtesting and compare the results

Strategy optimization example

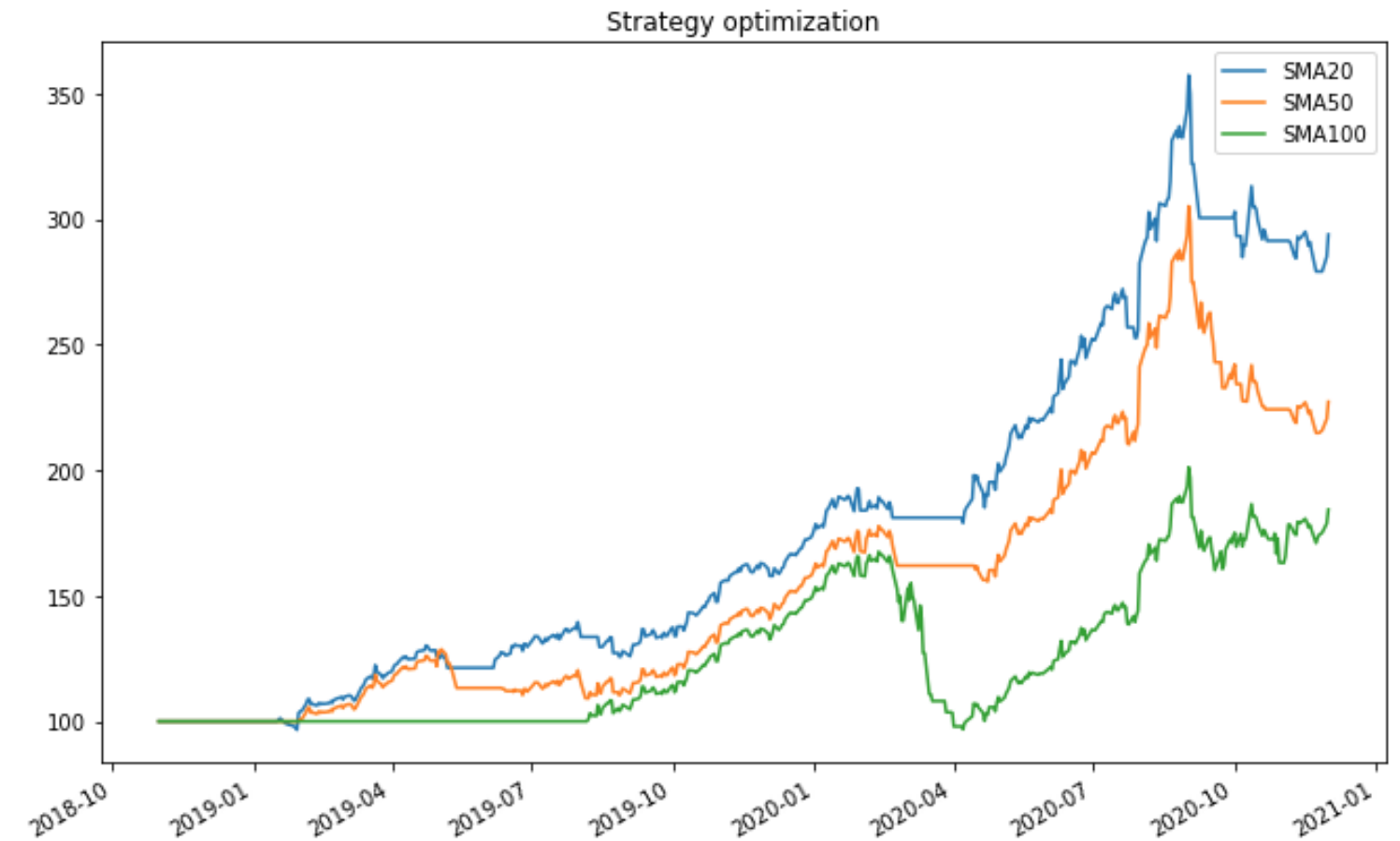
```
def signal_strategy(ticker, period, name, start='2018-4-1', end='2020-11-1'):  
    # Get the data and calculate SMA  
    price_data = bt.get(ticker, start=start, end=end)  
    sma = price_data.rolling(period).mean()  
    # Define the signal-based strategy  
    bt_strategy = bt.Strategy(name,  
                              [bt.algos.SelectWhere(price_data>sma),  
                               bt.algos.WeighEqually(),  
                               bt.algos.Rebalance()])  
  
    # Return the backtest  
    return bt.Backtest(bt_strategy, price_data)
```

¹ www.datacamp.com/courses/writing-functions-in-python

Strategy optimization example

```
ticker = 'aapl'
sma20 = signal_strategy(ticker,
                        period=20, name='SMA20')
sma50 = signal_strategy(ticker,
                        period=50, name='SMA50')
sma100 = signal_strategy(ticker,
                        period=100, name='SMA100')

# Run backtests and compare results
bt_results = bt.run(sma20, sma50, sma100)
bt_results.plot(title='Strategy optimization')
```



What is a benchmark?

A standard or point of reference against which a strategy can be compared or assessed.

- Example:
 - A strategy that uses signals to actively trade stocks can use a passive buy and hold strategy as a benchmark.
 - The S&P 500 index is often used as a benchmark for equities.
 - U.S. Treasuries are used for measuring bond risks and returns.

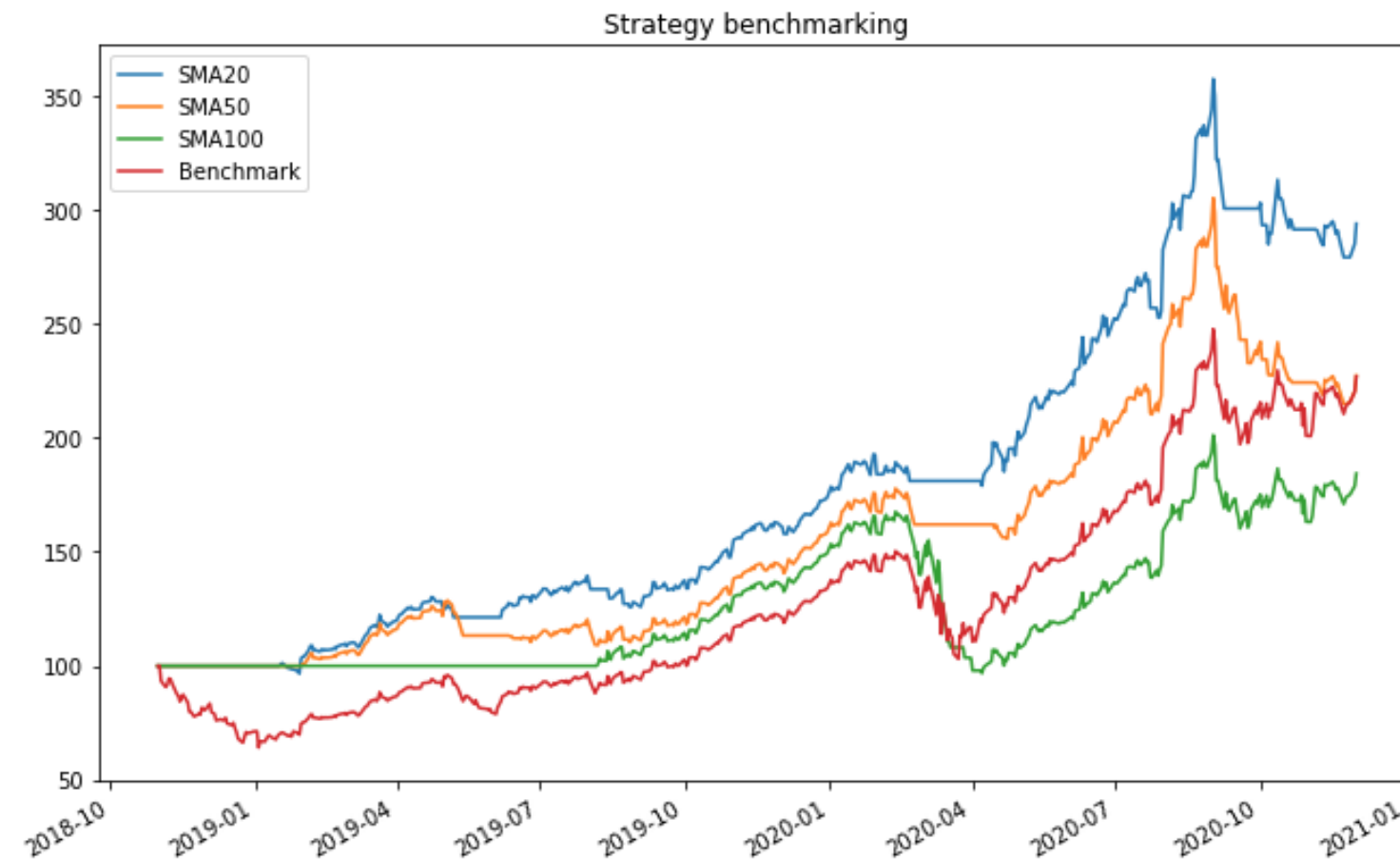
Benchmarking example

```
def buy_and_hold(ticker, name, start='2018-11-1', end='2020-12-1'):
    # Get the data
    price_data = bt.get(ticker, start=start_date, end=end_date)
    # Define the benchmark strategy
    bt_strategy = bt.Strategy(name,
                              [bt.algos.RunOnce(),
                               bt.algos.SelectAll(),
                               bt.algos.WeighEqually(),
                               bt.algos.Rebalance()])

    # Return the backtest
    return bt.Backtest(bt_strategy, price_data)
```

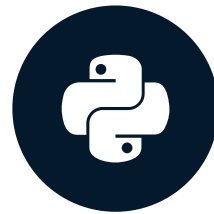
Benchmarking example

```
benchmark = buy_and_hold(ticker, name='benchmark')  
# Run all backtests and plot the results  
bt_results = bt.run(sma20, sma50, sma100, benchmark)  
bt_results.plot(title='Strategy benchmarking')
```



Strategy return analysis

FINANCIAL TRADING IN PYTHON



Chelsea Yang
Data Science Instructor

Obtain detailed backtest stats

```
# Get all backtest stats
resInfo = bt_result.stats
print(resInfo.index)
```

```
Index(['start', 'end', 'rf', 'total_return', 'cagr', 'max_drawdown', 'calmar',
      'mtd', 'three_month', 'six_month', 'ytd', 'one_year', 'three_year',
      'five_year', 'ten_year', 'incep', 'daily_sharpe', 'daily_sortino',
      'daily_mean', 'daily_vol', 'daily_skew', 'daily_kurt', 'best_day',
      'worst_day', 'monthly_sharpe', 'monthly_sortino', 'monthly_mean',
      'monthly_vol', 'monthly_skew', 'monthly_kurt', 'best_month',
      'worst_month', 'yearly_sharpe', 'yearly_sortino', 'yearly_mean',
      'yearly_vol', 'yearly_skew', 'yearly_kurt', 'best_year', 'worst_year',
      'avg_drawdown', 'avg_drawdown_days', 'avg_up_month', 'avg_down_month',
      'win_year_perc', 'twelve_month_win_perc'],
      dtype='object')
```

Strategy returns

$$Return = (V_e - V_b) / V_b$$

V_e : ending value

V_b : beginning value

```
# Get daily, monthly and yearly returns
print('Daily return: %.4f'% resInfo.loc['daily_mean'])
print('Monthly return: %.4f'% resInfo.loc['monthly_mean'])
print('Yearly return: %.4f'% resInfo.loc['yearly_mean'])
```

```
Daily return: 0.1966
Monthly return: 0.2207
Yearly return: 0.3328
```

Compound annual growth rate

$$CAGR = (V_f / V_i)^{\frac{1}{n}} - 1$$

V_f : final value

V_i : initial value

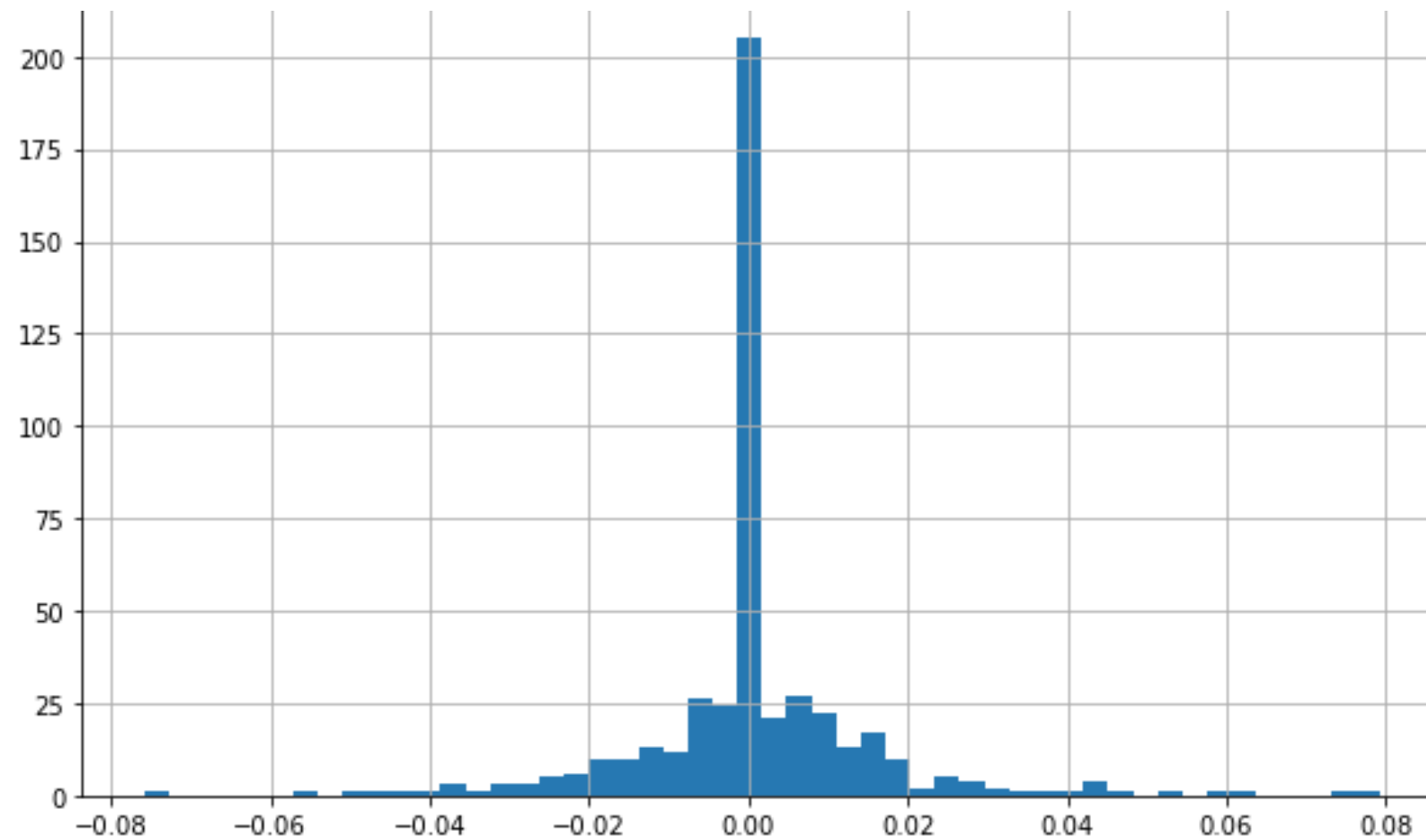
n : number of years

```
# Get the compound annual growth rate  
print('Compound annual growth rate: %.4f'% resInfo.loc['cagr'])
```

```
Compound annual growth rate: 0.1855
```

Plot return histogram

```
# Plot the weekly return histogram  
bt_result.plot_histograms(bins=50, freq = 'w')
```



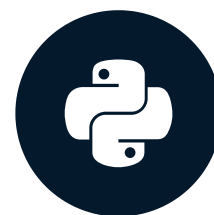
Compare strategy lookback returns

```
# Get the lookback returns
lookback_returns = bt_result.display_lookback_returns()
print(lookback_returns)
```

	Strategy1	Strategy2
mtd	3.30%	-0.03%
3m	0.68%	-2.15%
6m	8.11%	8.32%
ytd	28.08%	10.46%
1y	35.20%	17.09%
3y	11.48%	11.01%
5y	9.35%	9.48%
10y	9.35%	9.48%
incep	9.35%	9.48%

Drawdown

FINANCIAL TRADING IN PYTHON

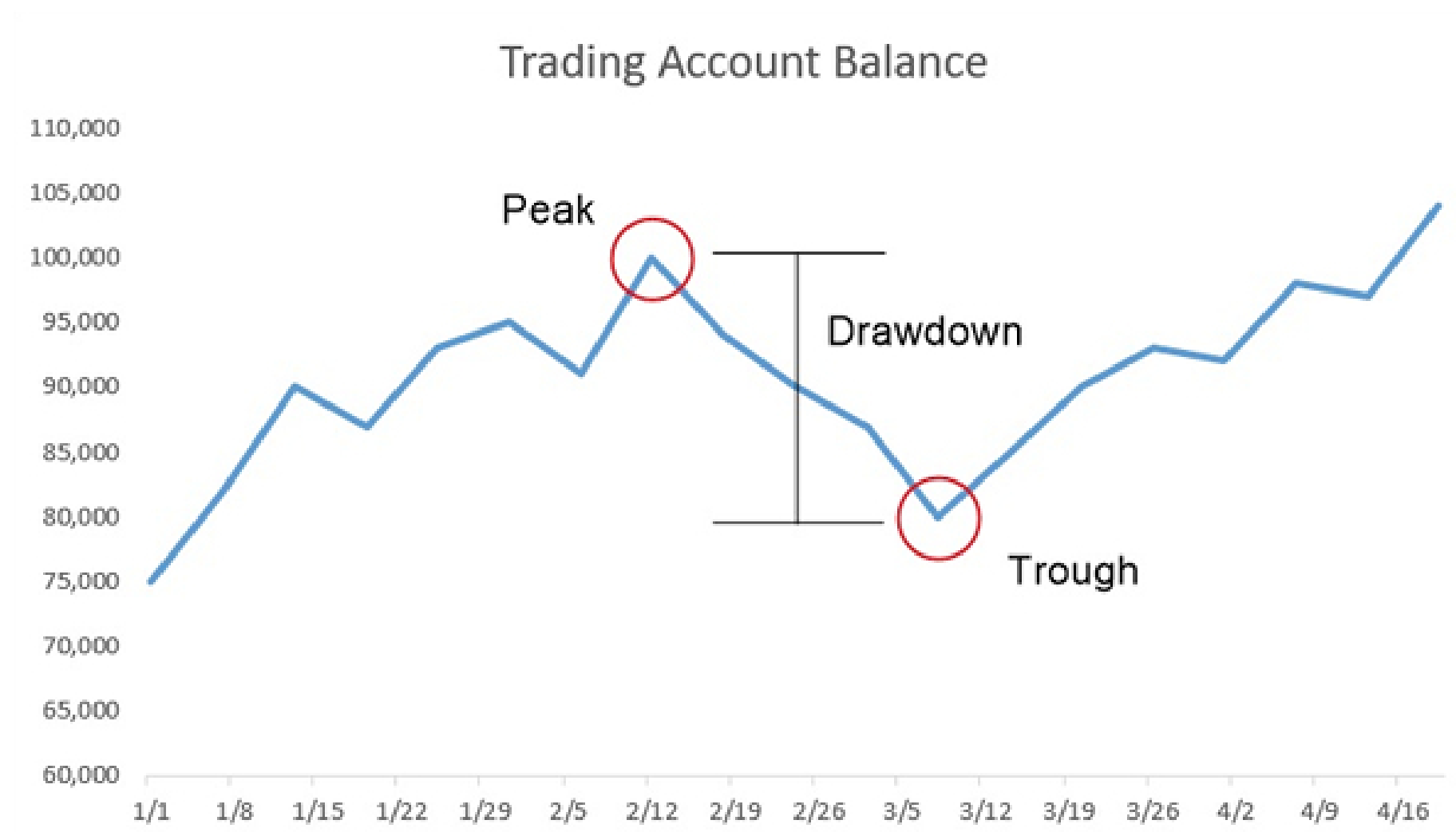


Chelsea Yang

Data Science Instructor

What is a drawdown?

A drawdown is a peak-to-trough decline during a specific period for an asset or a trading account.

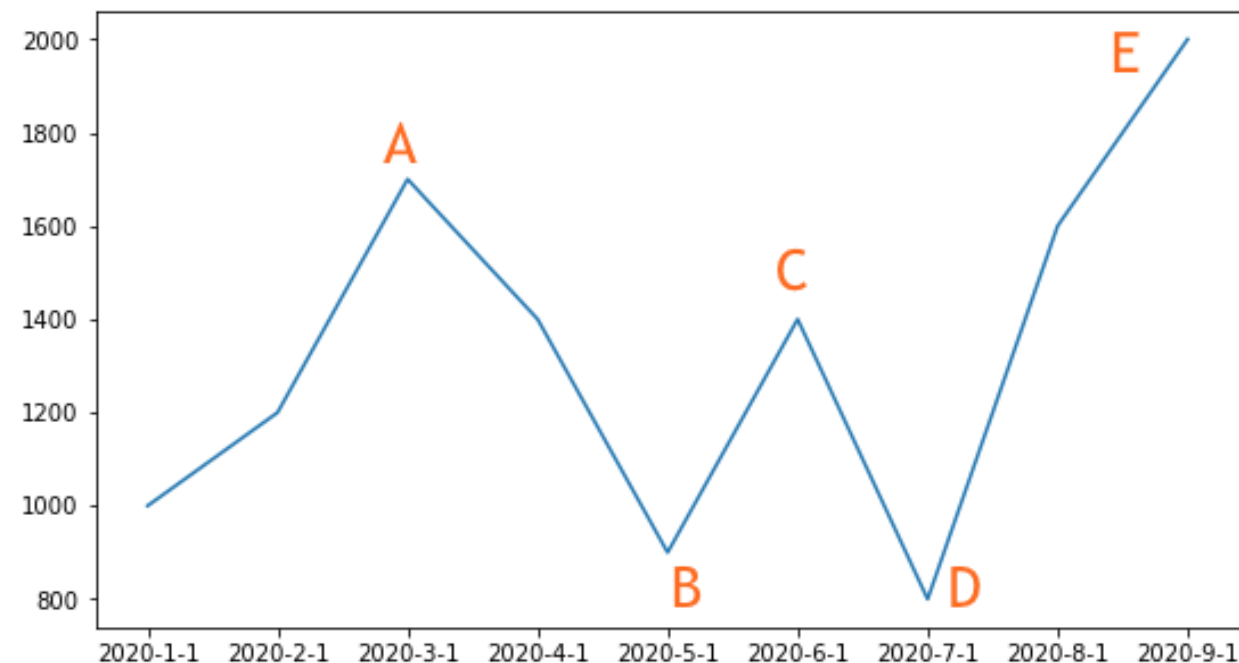


Max drawdown

$$\text{Max Drawdown} = (V_p - V_l) / V_l$$

V_p : Peak value before the largest drop

V_l : Lowest value before a new high value



Max drawdown

= (Point A value - point D value) / Point A value

= (1700 - 800) / 1700 = 53%

Obtain drawdowns from backtest stats

```
resInfo = bt_result.stats
# Get the max drawdown
max_drawdown = resInfo.loc['max_drawdown']
print('Maximum drawdown: %.2f'% max_drawdown)
# Get the average drawdown
avg_drawdown = resInfo.loc['avg_drawdown']
print('Average drawdown: %.2f'% avg_drawdown)
# Get the average drawdown days
avg_drawdown_days = resInfo.loc['avg_drawdown_days']
print('Average drawdown days: %.0f'% avg_drawdown_days)
```

```
Maximum drawdown: -0.59
Average drawdown: -0.11
Average drawdown days: 22
```

The Calmar ratio

CALMAR: CALifornia Managed Accounts Report

$$\text{Calmar} = \text{CAGR} / \text{Max Drawdown}$$

- The higher the Calmar ratio, the better a strategy performed on a risk-adjusted basis.
- Typically a Calmar ratio larger than 3 is considered excellent.

Calculate the Calmar ratio manually

```
resInfo = bt_result.stats
# Get the CAGR
cagr = resInfo.loc['cagr']
# Get the max drawdown
max_drawdown = resInfo.loc['max_drawdown']

# Calculate Calmar ratio manually
calmar_calc = cagr / max_drawdown * (-1)
print('Calmar Ratio calculated: %.2f' % calmar_calc)
```

```
Calmar Ratio calculated: 4.14
```

Obtain the Calmar ratio from backtest stats

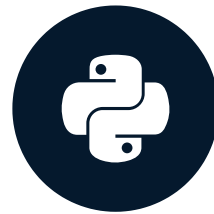
```
resInfo = bt_result.stats

# Get the Calmar ratio
calmar = resInfo.loc['calmar']
print('Calmar Ratio: %.2f'% calmar)
```

```
Calmar Ratio: 4.14
```

Sharpe ratio and Sortino ratio

FINANCIAL TRADING IN PYTHON



Chelsea Yang

Data Science Instructor

Which strategy performs better?

Strategy 1:

- Return: 15%
- Volatility (standard deviation): 30%

Strategy 2:

- Return: 10%
- Volatility (standard deviation): 8%

Risk-adjusted return

- Make performance comparable among different strategies
- A ratio that describes risk involved in obtaining the return



Sharpe ratio

$$\text{Sharpe Ratio} = (R_p - R_r) / \sigma_p$$

- R_p : Return of a strategy, portfolio, asset, etc.
- R_r : Risk-free rate
- σ_p : Standard deviation of the excess return ($R_p - R_f$)
- The bigger the Sharpe ratio, the more attractive the return

Now choose again

Strategy 1:

- Return: 15%
- Volatility (standard deviation): 30%
- Sharpe ratio: $15\%/30\% = 0.5$

Strategy 2:

- Return: 10%
- Volatility (standard deviation): 8%
- Sharpe ratio: $10\%/8\% = 1.25$

Obtain Sharpe ratio from bt backtest

```
resInfo = bt_result.stats
```

```
# Get Sharpe ratios from the backtest stats  
print('Sharpe ratio daily: %.2f'% resInfo.loc['daily_sharpe'])  
print('Sharpe ratio monthly %.2f'% resInfo.loc['monthly_sharpe'])  
print('Sharpe ratio annually %.2f'% resInfo.loc['yearly_sharpe'])
```

```
Sharpe ratio daily: 0.49  
Sharpe ratio monthly 0.48  
Sharpe ratio annually 1.34
```

Calculate Sharpe ratio manually

```
# Obtain annual return
annual_return = resInfo.loc['yearly_mean']

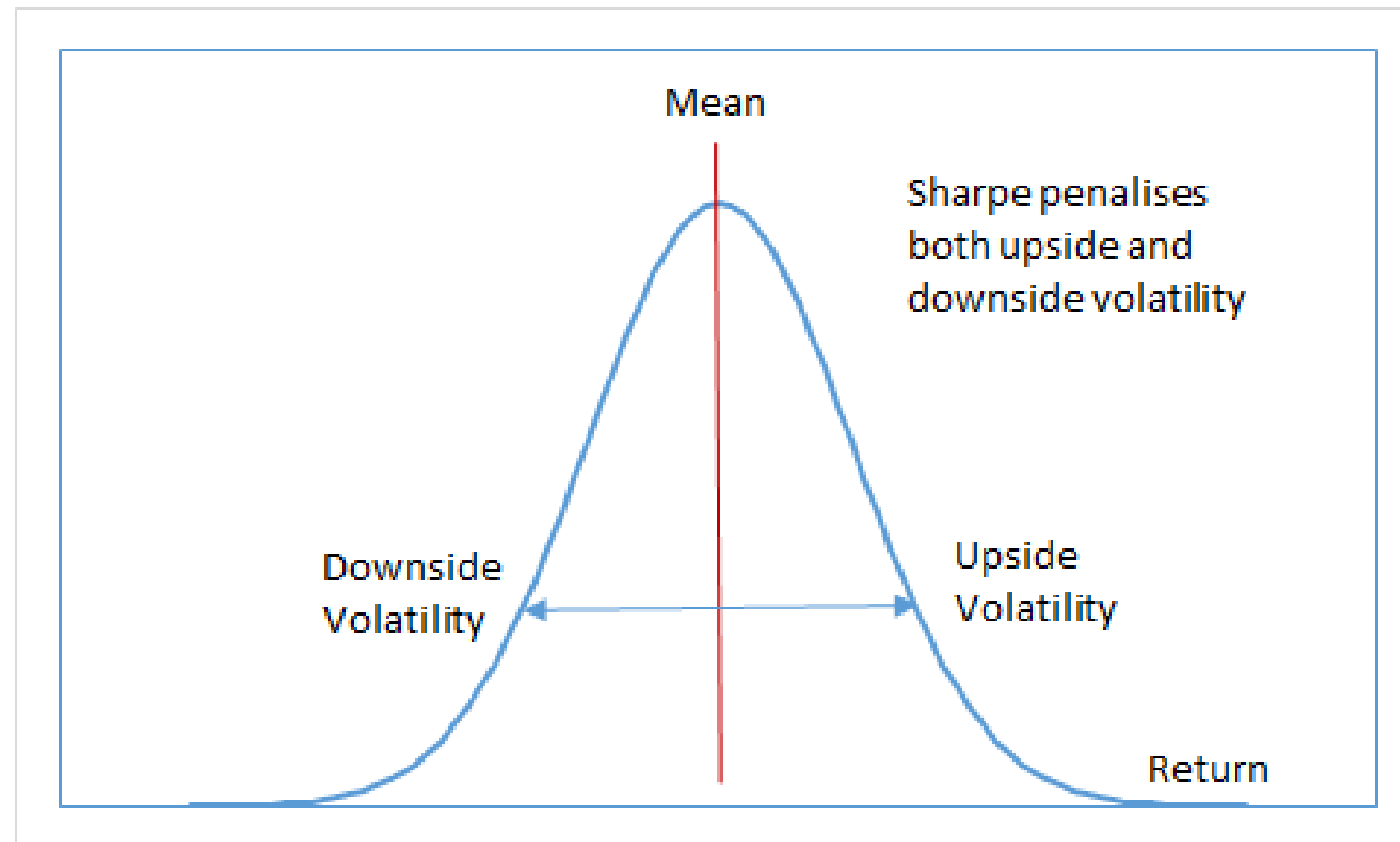
# Obtain annual volatility
volatility = resInfo.loc['yearly_vol']

# Calculate Sharpe ratio manually
sharpe_ratio = annual_return / volatility
print('Sharpe ratio annually %.2f'% sharpe_ratio)
```

```
Sharpe ratio annually 1.34
```

Limitations of Sharpe ratio

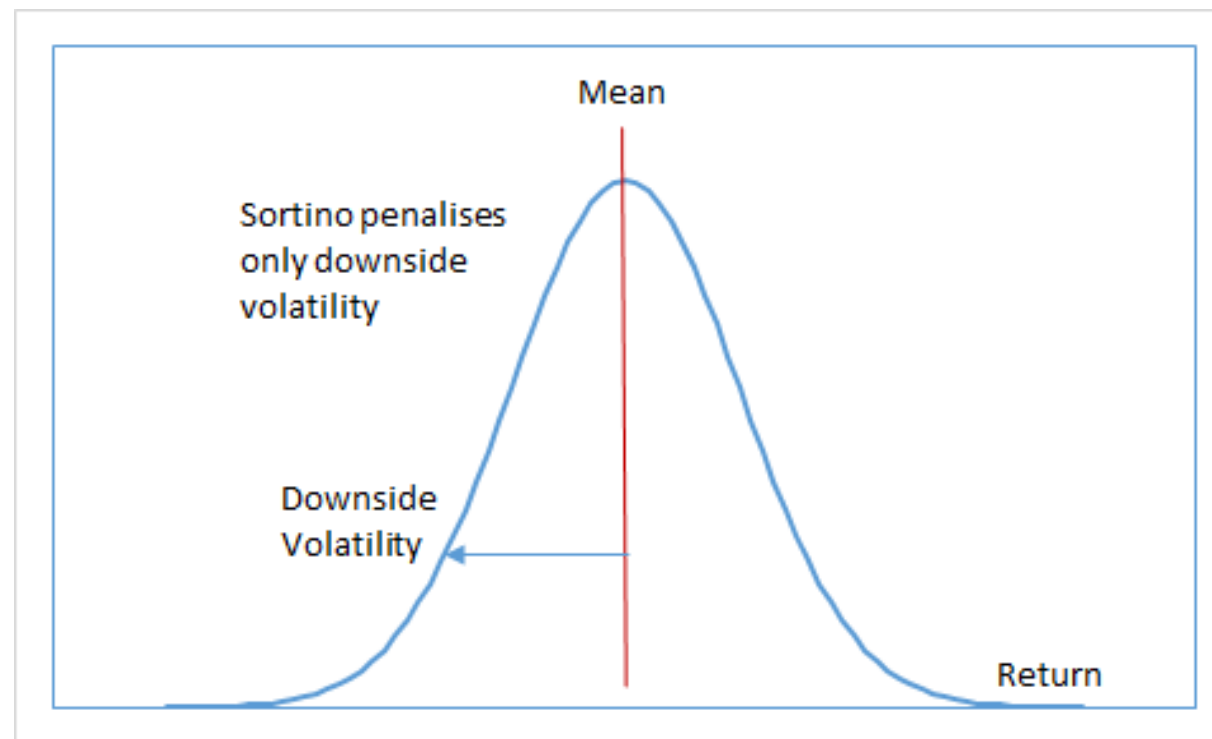
- Penalize both the "good" and "bad" volatility
- Upside volatility can skew the ratio downward



Sortino ratio

$$\text{Sortino Ratio} = (R_p - R_r) / \sigma_d$$

- R_p : Return of a strategy, portfolio, asset, etc
- R_r : Risk-free rate
- σ_d : Downside deviation of the excess return ($R_p - R_f$)



Obtain Sortino ratio from bt backtest

```
resInfo = bt_result.stats
```

```
# Get Sortino ratio from backtest stats  
print('Sortino ratio daily: %.2f'% resInfo.loc['daily_sortino'])  
print('Sortino ratio monthly %.2f'% resInfo.loc['monthly_sortino'])  
print('Sortino ratio annually %.2f'% resInfo.loc['yearly_sortino'])
```

```
Sortino ratio daily: 0.70  
Sortino ratio monthly 0.86  
Sortino ratio annually 2.29
```

You did it!

- Get trading data, check and plot the data
- Calculate technical indicators
- Build signal-based strategies and perform strategy backtesting
- Conduct strategy optimization and benchmarking
- Evaluate strategy performance